

PRODUCTION FOCUSED
PRACTICAL EXAMPLES
TROUBLESHOOTING GUIDES
ARCHITECTURE INSIGHTS
SENIOR ENGINEER PERSPECTIVE

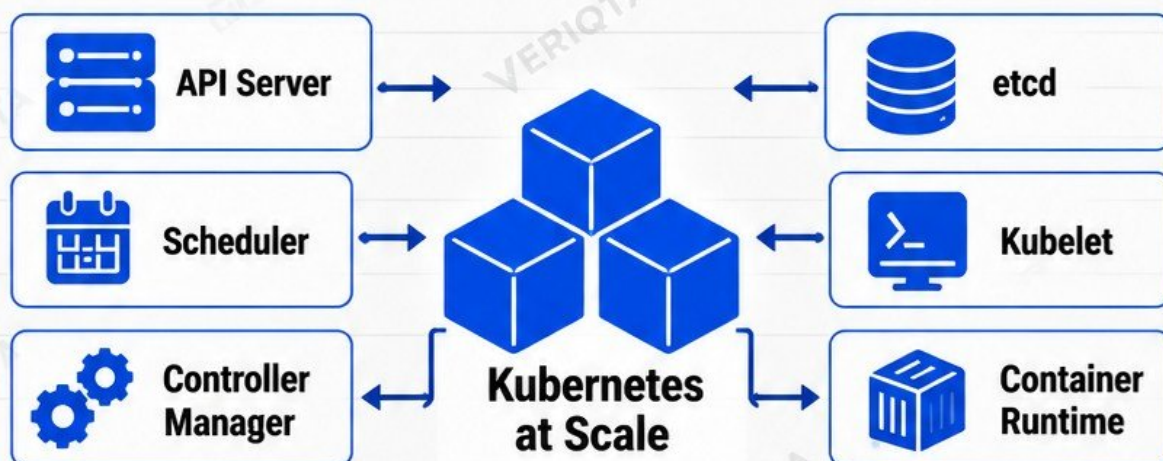


REAL-WORLD
KNOWLEDGE
FOR MODERN
PLATFORMS

Kubernetes BLACK BOOK

A 20-PAGE SENIOR ENGINEER GUIDE

ARCHITECTURE | OPERATIONS | TROUBLESHOOTING | SECURITY | SCALING



REAL-WORLD
SCENARIOS



PRACTICAL
COMMANDS



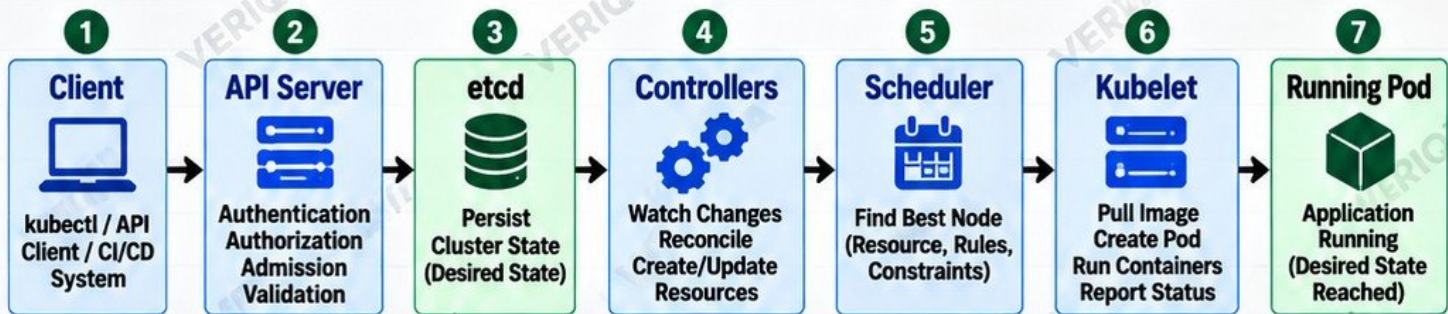
PRODUCTION
INSIGHTS



ENGINEERING
BEST PRACTICES

Kubernetes Request Path: From Client to Running Pod

UNDERSTAND THE FULL JOURNEY, KNOW WHERE IT CAN FAIL, AND HOW TO DEBUG IT



1. API REQUEST FLOW

- Client sends request using kubectrl, API, or another tool.
- Request goes to kube-apiserver.
- API server processes, validates and persists the change in etcd.
- Controllers detect the change and reconcile the desired state.
- Scheduler assigns the Pod to a node.
- Kubelet pulls the image and creates the container.
- Pod transitions to Running when all conditions are met.

2. AUTHENTICATION, AUTHORIZATION, ADMISSION

- Authentication:** Verify identity (certificates, tokens, OIDC).
- Authorization:** Check permissions (RBAC, ABAC, webhook).
- Admission:** Validate and mutate requests (e.g. policies, security, resource limits).
- If any step fails, the request is rejected before it reaches etcd.

3. ETCD PERSISTENCE

- Kubernetes stores all cluster state in etcd.
- Every successful API request is written to etcd.
- If etcd is unavailable, the cluster cannot accept changes.
- Data is stored as key-value pairs.
- etcd uses Raft for consistency and high availability.

4. CONTROLLERS AND RECONCILIATION

- Controllers watch for changes in etcd.
- They compare desired state vs actual state.
- If the actual state does not match, they take action to fix it.
- Examples: Deployment, ReplicaSet, Node, Endpoint, Service controllers.
- Reconciliation is continuous and eventual.
- If a controller is down, desired state may not be enforced.

5. SCHEDULER AND KUBELET INTERACTION

- Scheduler watches for unscheduled Pods.
- Evaluates nodes based on resources, taints, tolerations, affinity, topology and policies.
- Selects the best node and updates the Pod in etcd.
- Kubelet on the node sees the assigned Pod.
- Kubelet pulls the image, creates the container, runs probes and reports status back to API server.

6. WHERE FAILURES APPEAR ALONG THE PATH

- Request rejected (auth, RBAC, admission).
- etcd unavailable or slow.
- Controller not reconciling.
- Pod stuck in Pending (no suitable node).
- Image pull failures.
- Kubelet issues (disk, network, permissions).
- Pod stuck in ContainerCreating, CrashLoopBackOff or NotReady.

COMMON FAILURE SYMPTOMS

STAGE	SYMPTOM
Client	Connection errors
API Server	401, 403, 429 errors
Admission	Request denied
etcd	Timeouts, high latency
Controller	Desired state not reconciled
Scheduler	Pod stuck in Pending
Kubelet	Image pull or runtime errors
Pod	CrashLoopBackOff, NotReady

USEFUL COMMANDS

```

# Check API server
kubectrl get --raw /healthz

# Check events
kubectrl get events --sort-by=.lastTimestamp

# Describe pod
kubectrl describe pod <pod-name>

# Check controller status
kubectrl get deployment <name> -o wide

# Check node condition
kubectrl describe node <node-name>
  
```



PRODUCTION NOTE

Most Kubernetes issues are not in your application. They are in the control plane, scheduling, networking, image pulls or node health.



SENIOR ENGINEER TIP

When debugging, always determine which stage the failure is in. It reduces investigation time and avoids random troubleshooting.

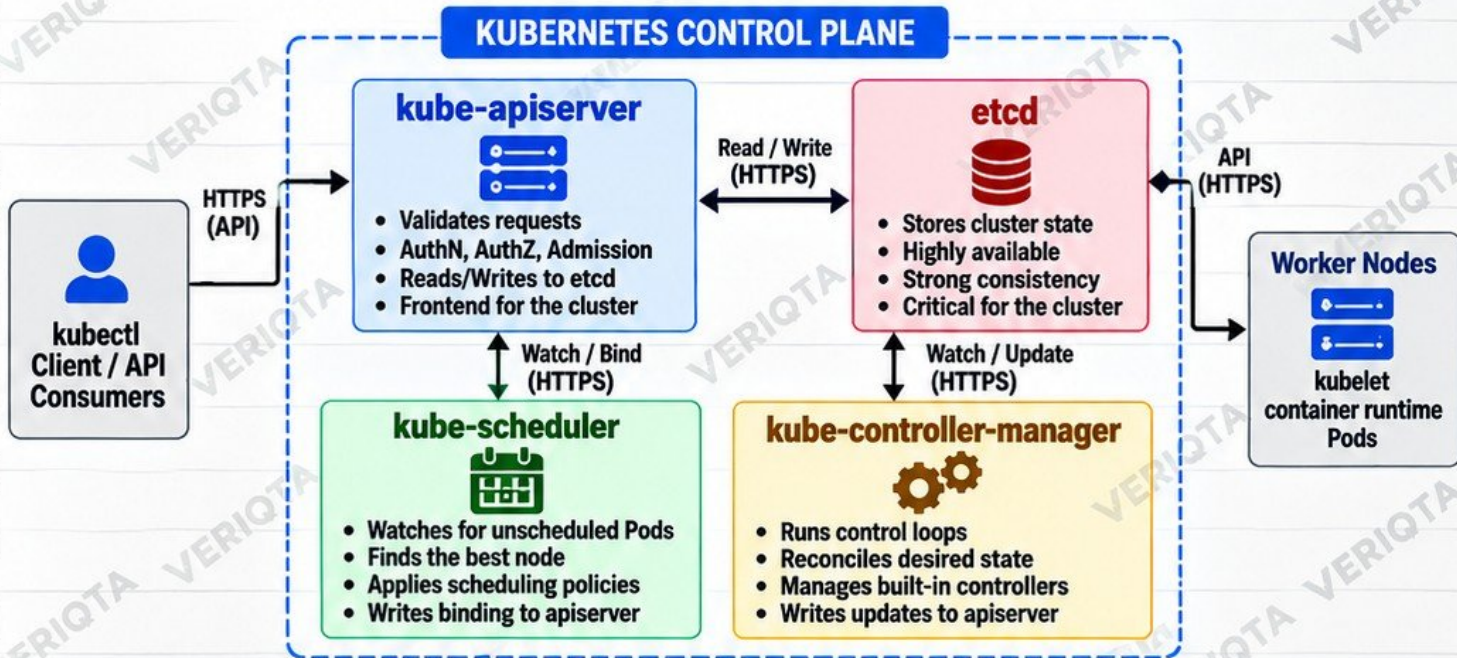


GOAL

Understand the full request path so you can quickly identify where a Kubernetes issue is coming from and take the right action.

Control Plane Under the Hood

COMPONENTS, HOW THEY WORK TOGETHER, FAILURE BEHAVIOR AND RECOVERY



1. KUBE-APISERVER

- Entry point for all requests (kubectl, UI, controllers).
- Handles authentication, authorization and admission.
- Validates and persists changes to etcd.
- Exposes REST API used by all components.
- Typically runs in multiple replicas for high availability.
- If it is down, you cannot manage the cluster.

2. ETCD

- Distributed key-value store for all cluster state.
- Stores objects like Pods, Deployments, ConfigMaps, etc.
- Uses Raft for consensus and high availability.
- Requires odd number of nodes (3 or 5).
- If etcd is unavailable or loses quorum, the cluster cannot persist changes and may become read-only or unavailable.

3. KUBE-SCHEDULER

- Watches for Pods without a node.
- Filters and scores nodes based on resources, taints, tolerations, affinity/anti-affinity, topology, and policies.
- Binds the Pod to the selected node by updating the apiserver.
- Can run multiple replicas with leader election.
- If it is down, new Pods will remain in Pending state.

4. KUBE-CONTROLLER-MANAGER

- Runs controllers (Deployment, ReplicaSet, Node, Endpoint, Service, etc.).
- Continuously reconciles actual state with desired state.
- If something is not in the desired state, it takes action to fix it.
- Multiple replicas run with leader election (only one active).
- If it is down, objects will not be reconciled (e.g. failed nodes not replaced, replicas not created).

5. LEADER ELECTION

- Ensures only one active instance of scheduler and controller-manager.
- Uses a Lease object in etcd.
- If the leader fails, another replica automatically takes over.
- Prevents split-brain and duplicate actions.
- Depends on healthy etcd and apiserver.

6. CONTROL-PLANE DEPENDENCIES

- Apiserver depends on etcd.
- Controllers and scheduler depend on apiserver.
- All components rely on networking, certificates, DNS and time sync (NTP).
- If etcd or apiserver is unavailable, the whole control plane is affected.

7. FAILURE BEHAVIOR AND RECOVERY

COMPONENT	FAILURE BEHAVIOR	RECOVERY
kube-apiserver	Cannot manage cluster, API errors	Fix process, check certs, network, restart or failover
etcd	Cluster may become read-only or down	Restore from backup or recover quorum
kube-scheduler	New Pods stay Pending	Check leader election, restart
kube-controller-manager	No reconciliation (e.g. replicas not created)	Check leader election, restart
All components	If etcd or apiserver down, cluster functionality is severely impacted	Restore etcd, bring up apiserver, verify components, check health

⚠️ PRODUCTION LESSON

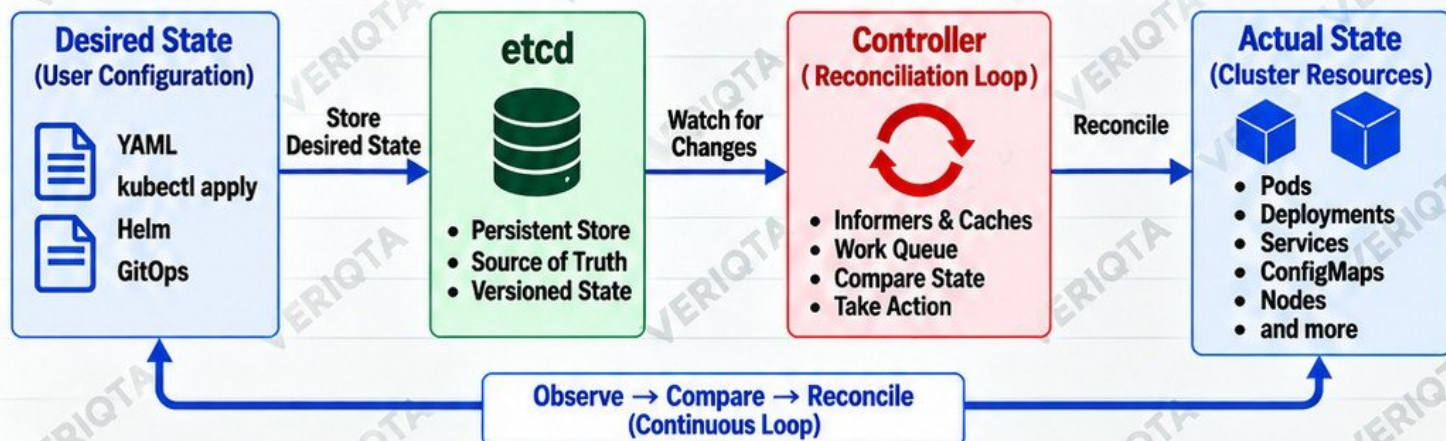
A Kubernetes cluster is only as healthy as its control plane. Always monitor etcd health, apiserver latency and component leader status. Loss of etcd or apiserver can quickly take down the entire cluster.

💡 SENIOR ENGINEER TIP

Keep multiple control-plane replicas, monitor etcd quorum, set up proper alerting, and regularly test recovery from control-plane failures.

Kubernetes Reconciliation: The Engine Behind the Cluster

HOW KUBERNETES CONTROLLERS TURN DESIRED STATE INTO REALITY



1. DESIRED VS OBSERVED STATE

- Desired state = what you define (YAML).
- Observed state = what actually exists in the cluster.
- Controllers constantly compare the two states.
- If they do not match, the controller takes action.
- This applies to Deployments, ReplicaSets, Nodes, Services and all built-in and custom controllers.

2. CONTROL LOOPS

- Kubernetes uses continuous control loops.
- Each controller watches for changes.
- When a difference is detected, it reconciles the state.
- The loop runs continuously, not just once.
- This makes the system self-healing and eventually consistent.

3. INFORMERS, WATCHES AND CACHES

- Controllers use informers to watch the API server for changes.
- A local cache stores objects to reduce API calls.
- Watches provide real-time updates.
- If the watch connection is lost, it falls back to relisting.
- Caching improves performance and scales to large clusters.

4. WORK QUEUES

- When a change is detected, a key (e.g. a resource name) is added to a work queue.
- The controller processes items from the queue and reconciles the state.
- Rate limiting prevents overloaded controllers.
- Failed items are retried with backoff.
- This ensures reliability even during high change volume.

5. EVENTUAL CONVERGENCE

- Kubernetes guarantees eventual convergence, not immediate consistency.
- There may be a short delay between a change and the desired state being reached.
- During this time, the system continues to reconcile.
- Convergence depends on controller health, API server availability and successful actions (e.g. creating a Pod).

6. STUCK RECONCILIATION

- Can happen when the controller cannot complete the desired action.
- **Common causes:** insufficient permissions, invalid configuration, resource quotas, image pull failures, or missing dependencies.
- **Symptoms:** object remains in a non-ready state, repeated events, or no progress.
- Check controller logs, events and the work queue for errors.

7. CONTROLLER FAILURE PATTERNS

- Controller is down or crashing.
- Lost leader election (no active controller).
- Cannot access the API server or etcd.
- Permission or RBAC errors.
- External dependency failures (e.g. cloud API).
- Logic bugs or unexpected object state.
- Results in resources not being reconciled, which can lead to outages or drift.



KEY TAKEAWAY

Reconciliation is what makes Kubernetes reliable. Even when components fail, the control loops keep working to bring the cluster back to the desired state.

“DEFINE. OBSERVE. RECONCILE. REPEAT.”



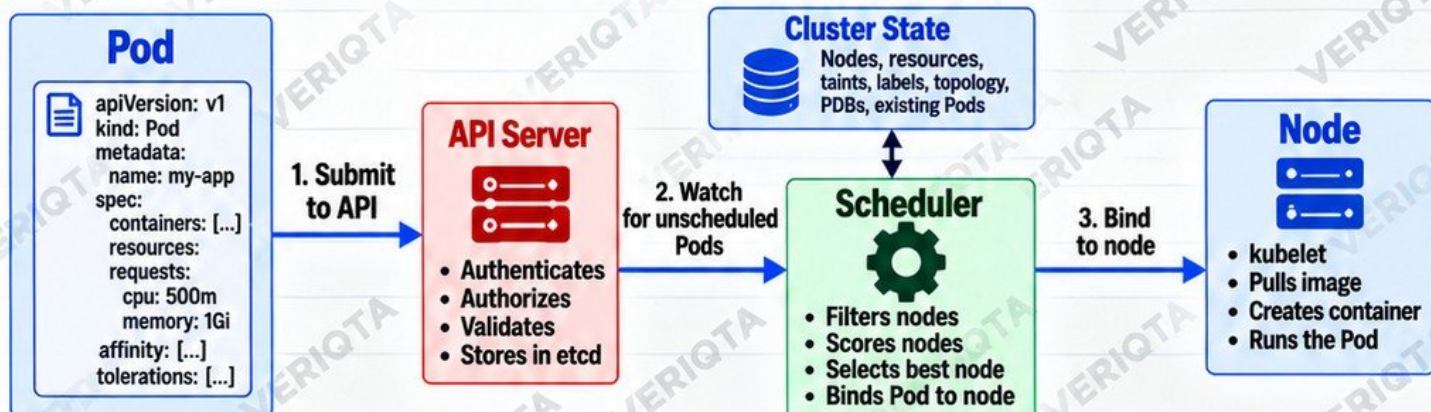
SENIOR ENGINEER TIP

When troubleshooting, always check:

1. Is the controller running?
2. Are there errors in the logs?
3. Are events showing failures?
4. Does the controller have the required permissions?
5. Are there external dependencies blocking progress?

Kubernetes Scheduling: Why Your Pod Is Still Pending

HOW THE SCHEDULER DECIDES, WHAT CAN BLOCK SCHEDULING,
AND HOW TO INVESTIGATE PENDING PODS



1. REQUESTS AND SCHEDULING FEASIBILITY

- Scheduler checks if a node has enough free CPU, memory, GPU and other resources.
- It uses resource requests (not limits) for scheduling decisions.
- If no node has enough available resources, the Pod remains Pending.
- Overcommitted clusters or large requests can make Pods unschedulable.

2. NODE SELECTORS AND AFFINITY

- Node selectors require exact label matches.
- Node affinity is more flexible (required or preferred).
- Used to place Pods on specific node types (e.g. GPU, high-memory, edge).
- If no nodes match the rules, the Pod stays Pending.

3. TAINTS AND TOLERATIONS

- Taints repel Pods from nodes.
- Pods must have matching tolerations to be scheduled on tainted nodes.
- Common taints: node-role.kubernetes.io/master, dedicated, custom taints.
- If tolerations are missing, the Pod remains Pending even if resources are available.

4. TOPOLOGY SPREAD CONSTRAINTS

- Ensures Pods are spread across zones, nodes or other topology domains.
- Prevents all replicas from being placed in the same failure domain.
- If constraints cannot be satisfied, the Pod may remain Pending.
- Check topologyKeys, maxSkew and whenUnsatisfiable settings.

5. PREEMPTION

- If a high-priority Pod cannot be scheduled, the scheduler may preempt lower-priority Pods.
- Depends on PriorityClass and Pod Disruption Budget (PDB).
- Preemption helps critical workloads but can cause disruption.
- If preemption is not possible, the Pod stays Pending.

6. SCHEDULER BOTTLENECKS

- Large clusters with many Pods can slow scheduling.
- Complex affinity, topology and PDB rules increase scheduling time.
- Watch for high scheduler CPU, memory or long scheduling latency.
- Scheduler leader election issues or unavailable scheduler can stop scheduling.

7. PENDING POD INVESTIGATION

- Check Pod events: `kubectl describe pod <pod-name>`
- Look for scheduling errors and reasons.
- Check cluster resources: `kubectl get nodes`
- Verify labels, taints, tolerations and affinity.
- Review scheduler logs: `kubectl logs -n kube-system -l component=kube-scheduler`
- Check for PDB blocking, quota limits or topology constraints.
- Confirm the scheduler is healthy and leader is active.

COMMON PENDING REASONS

- Insufficient CPU or memory
- Node selector mismatch
- Toleration not found
- Topology spread constraints not satisfied
- PDB blocking
- No available nodes
- Scheduler not running or high latency



SENIOR ENGINEER TIP

Always start with `kubectl describe pod`. The Events section tells you exactly why the Pod is Pending.

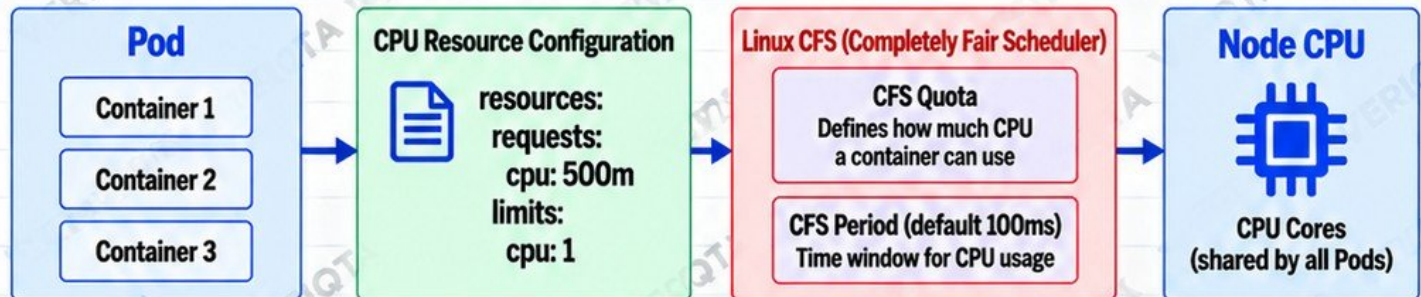


GOAL

Understand how the scheduler makes decisions so you can quickly identify and fix Pending Pods.

CPU: Requests, Limits and Throttling

HOW KUBERNETES MANAGES CPU, WHY THROTTLING HAPPENS,
AND HOW TO DEBUG PERFORMANCE ISSUES



1. CPU REQUESTS

- Guarantees a minimum amount of CPU for the container.
- Used by the scheduler to decide which node can run the Pod.
- Does not limit CPU usage, it only reserves capacity.
- Example: `cpu: 500m` = 0.5 vCPU.
- If total requests on a node exceed its capacity, Pods remain Pending.

2. CFS QUOTA BEHAVIOR

- Kubernetes uses Linux CFS (Completely Fair Scheduler) to enforce CPU limits.
- CFS quota defines how much CPU time a container can use in each period (default 100ms).
- Example: `limit 1 CPU` = 100ms of CPU time per 100ms period.
- If the container uses its quota early, it is throttled for the rest of the period.

3. CPU LIMITS

- Sets the maximum amount of CPU a container can use.
- Enforced using CFS quota.
- Prevents a container from consuming all node CPU.
- If no limit is set, the container can use more CPU (up to node capacity).
- Example:
`requests: 500m`
`limits: 1`

4. THROTTLING

- Happens when a container exceeds its CPU limit (CFS quota).
- The process is paused until the next CFS period.
- Shows up as high `cpu throttling` metrics but not high CPU usage.
- Can cause increased latency and poor application performance.
- Check metrics: `container_cpu_cfs_throttled_seconds_total`

5. BURSTABLE WORKLOADS

- If `request < limit` → Burstable QoS class.
- Can use more CPU when available, but is throttled at the limit.
- Common for web applications and microservices.
- Good balance between performance and resource control.
- Be careful: CPU spikes can cause throttling and latency.

6. LATENCY CONSEQUENCES

- CPU throttling can increase request latency and response time.
- Applications with tight SLAs (e.g. APIs) are sensitive to throttling.
- Can cause timeouts, queue buildup and cascading failures.
- Even with low CPU usage, high throttling can still degrade performance.
- Monitor latency (`p95`, `p99`) alongside throttling metrics.

7. EVIDENCE-DRIVEN CPU DEBUGGING

```

# Check CPU usage
kubectl top pod <pod-name>

# Check throttling metrics
kubectl get --raw "/apis/metrics.k8s.io/v1beta1/nodes/" | jq

# Describe pod for events
kubectl describe pod <pod-name>

# Inspect container metrics (Prometheus example)
rate(container_cpu_cfs_throttled_seconds_total[5m])
  
```

COMMON SIGNS OF CPU THROTTLING

- High throttling metrics
- Normal or low CPU usage
- Increased application latency
- Slow request processing
- Intermittent timeouts
- No errors in application logs



SENIOR ENGINEER TIP

Set requests based on real usage, not guesses. Monitor throttling and latency in production and adjust limits if needed.



BEST PRACTICES

- Set realistic requests and limits.
- Monitor CPU throttling metrics.
- Use autoscaling for variable workloads.
- Test under production-like load.
- Avoid overcommitting nodes.
- Consider latency SLOs when setting limits.

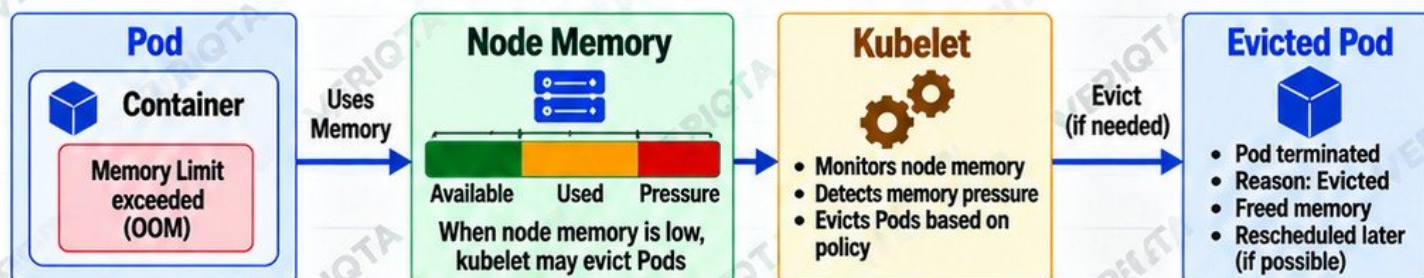


GOAL

Understand how CPU requests, limits and CFS quota work, detect throttling early, and optimize for performance and stability in production.

Memory Pressure, OOMKills and Eviction

HOW KUBERNETES HANDLES MEMORY, WHAT CAUSES OOMKILLS AND EVICTIONS,
AND HOW TO INVESTIGATE AND FIX MEMORY-RELATED ISSUES



1. MEMORY REQUESTS AND LIMITS

- Requests = minimum memory guaranteed for the container.
- Limits = maximum memory a container can use.
- If a container uses more than its limit, it can be OOMKilled.
- Requests help with scheduling.
- Limits prevent a single container from consuming all node memory.
- Always set realistic requests and limits based on actual usage.

Example:

```
resources:
  requests:
    memory: "256Mi"
  limits:
    memory: "512Mi"
```

2. CGROUPS

- Kubernetes uses Linux cgroups to enforce memory limits.
- Cgroups track and limit the memory usage of each container.
- When a container exceeds its memory limit, the kernel OOM killer terminates it.
- Memory usage is isolated per container using cgroups.

3. CONTAINER OOM

- Occurs when a container uses more memory than its limit.
- The Linux kernel OOM killer terminates the container process.
- Pod status: OOMKilled
- The Pod may restart based on restartPolicy.
- Check logs and metrics to identify the memory spike.

Check OOMKilled Pods:

```
kubectl get pods -A | grep OOMKilled
```

4. NODE MEMORY PRESSURE

- Happens when the node is low on available memory (affects all Pods).
- Kubelet sets node condition: MemoryPressure.
- Can be caused by:
 - High memory usage from Pods
 - System daemons
 - Large page cache
 - Memory leaks
 - Too many Pods on the node
- When memory pressure is detected, kubelet may evict Pods to free memory.

Check node memory pressure:

```
kubectl describe node <node-name> | grep -i memory
kubectl get nodes
```

5. KUBELET EVICTION

- If the node is under memory pressure, kubelet evicts Pods based on eviction thresholds.
- Eviction is proactive to protect the node.
- Pods are terminated with reason: Evicted.
- Eviction can happen even if the Pod is within its memory limit.
- Check kubelet logs for eviction events.

8. PRODUCTION INVESTIGATION FLOW

- 1 Check Pod status and events
`kubectl describe pod <pod-name>`
- 2 Check container logs (look for OOM errors)
- 3 Check resource usage (metrics)
- 4 Check node memory pressure
`kubectl describe node <node-name>`
- 5 Review kubelet logs (evictions)
- 6 Identify memory spikes (application logs, metrics)
- 7 Fix root cause (increase limit, optimize app, reduce memory usage, right-size nodes)



GOAL

Understand how Kubernetes handles memory, quickly identify whether a Pod was OOMKilled or evicted, and apply the right fix to prevent recurrence.

6. QOS CLASSES

Kubernetes assigns a QoS class to each Pod based on its requests and limits.

QoS Class	Requests	Limits	Behavior
Guaranteed	Set	Set (same)	Least likely to be evicted
Burstable	Set	Higher	Can be evicted under pressure
BestEffort	Not set	Not set	Most likely to be evicted



COMMON SIGNS

- Pod status: OOMKilled or Evicted
- Node condition: MemoryPressure
- Frequent Pod restarts
- Increasing memory usage over time
- Application crashes or slow performance

7. OOMKILLED VS EVICTION

Aspect	OOMKilled	Evicted
Cause	Container exceeded memory limit	Node under memory pressure
Who terminates	Linux kernel (inside container)	Kubelet (on the node)
Reason	OOMKilled	Evicted
Scope	Single container	Whole Pod
Node memory pressure	Not required	Required

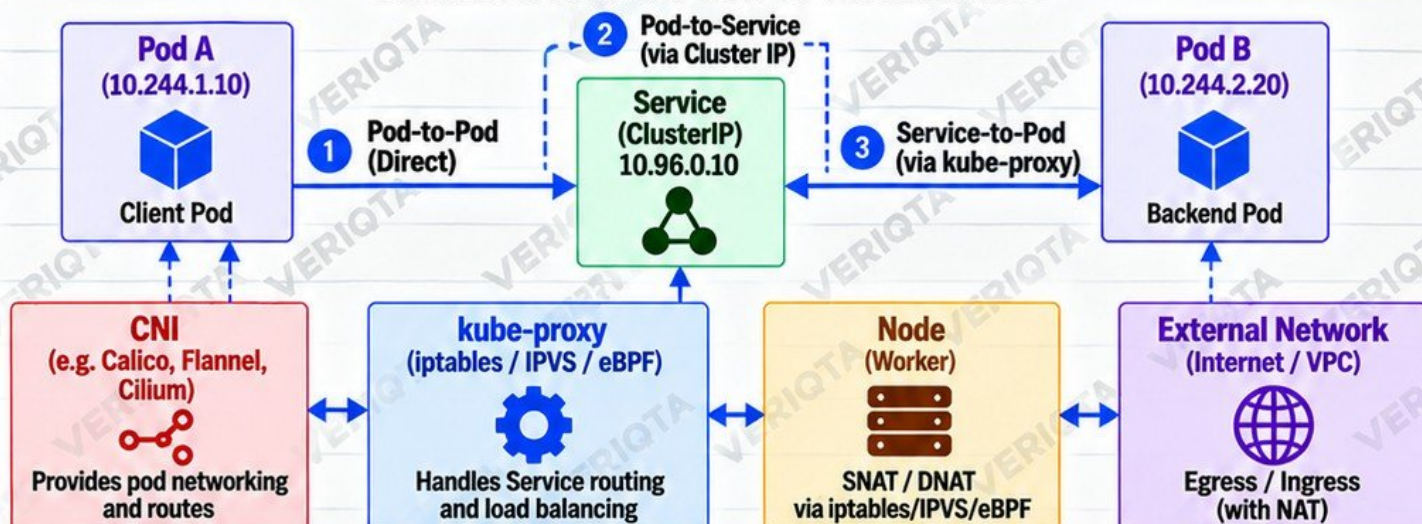


SENIOR ENGINEER TIP

- Always monitor memory usage and set proper requests and limits.
- Use Prometheus and Grafana for memory alerts.
- Analyze patterns, not just single spikes.
- Right-size your nodes and avoid overcommitment.
- Investigate and fix the root cause, not just increase limits.

Kubernetes Networking: Packet Journey

HOW PACKETS TRAVEL IN KUBERNETES, KEY COMPONENTS,
COMMON ISSUES AND HOW TO TROUBLESHOOT



1. POD-TO-POD

- Pods communicate directly using their Pod IP addresses.
- Handled by the CNI plugin (e.g. Calico, Flannel, Cilium).
- Requires the network to allow traffic (policies, security groups).
- If it fails, check CNI, NetworkPolicy and routing.

4. CNI (CONTAINER NETWORK INTERFACE)

- Sets up Pod networking (IP allocation, routing, overlay/underlay).
- Common CNIs: Calico, Flannel, Cilium, Weave.
- Handles Pod-to-Pod communication across nodes.
- If it fails, Pods cannot reach other Pods or Services.

7. SNAT / DNAT

- SNAT (Source NAT): Changes the source IP (usually for outbound traffic).
- DNAT (Destination NAT): Changes the destination IP (used for Services).
- Handled by iptables, IPVS or eBPF.
- Misconfigured NAT can break connectivity (e.g. external access, service routing).

2. POD-TO-SERVICE

- Pod sends traffic to the Service ClusterIP.
- kube-proxy intercepts the traffic and forwards it to a healthy Pod (Endpoint).
- Load balancing is done at the node (using iptables, IPVS or eBPF).
- If it fails, check kube-proxy, endpoints and Service configuration.

5. KUBE-PROXY

- Runs on every node.
- Implements Service routing using iptables, IPVS or eBPF.
- Tracks Endpoints and updates rules.
- If it fails, Services will not route traffic to Pods.
- Check kube-proxy logs and mode (iptables/IPVS/eBPF).

8. WHERE PACKETS DISAPPEAR

- Common causes:
 - NetworkPolicy blocking traffic
 - CNI misconfiguration or failure
 - kube-proxy not updating rules
 - Pod not Ready or missing endpoints
 - Firewall / security group rules
 - SNAT/DNAT issues
 - MTU problems (packet fragmentation)
 - Node routing issues

3. SERVICE-TO-POD

- kube-proxy selects a healthy backend Pod from EndpointSlices.
- Traffic is forwarded to the Pod IP.
- If the Pod is not Ready, it will not receive traffic.
- If it fails, check endpoints, readiness probes and NetworkPolicy.

6. IPTABLES / IPVS / eBPF CONTEXT

- iptables: Default mode, simple but can become slow at scale.
- IPVS: Better performance for large clusters.
- eBPF: Used by modern CNIs (e.g. Cilium) for high performance and observability.
- All handle Service routing and load balancing differently.
- Check which mode is in use and verify rules or maps.



SENIOR ENGINEER TIP

- Use a systematic approach:
 - Check Pod and Service configuration
 - Verify endpoints and kube-proxy rules
 - Check CNI and node network
 - Test with tools (ping, curl, tcpdump, conntrack)
 - Inspect NetworkPolicy and firewall rules
 - Follow the packet step by step



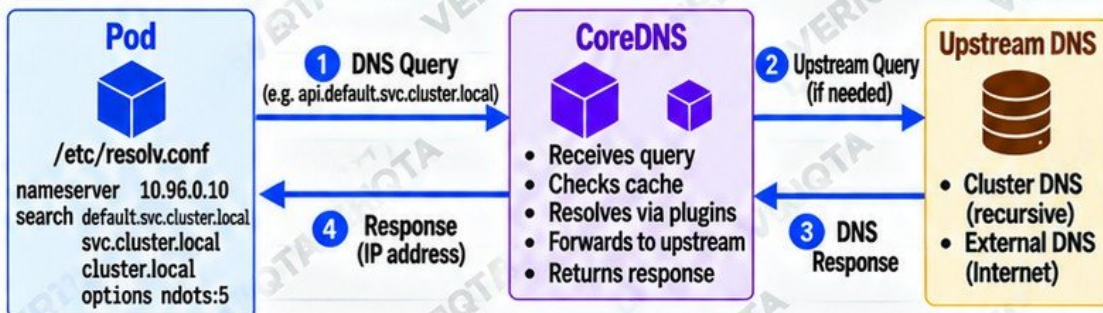
GOAL

Understand the full packet journey in Kubernetes and quickly identify where packets are dropped.

Kubernetes DNS Failure Anatomy

HOW DNS WORKS IN KUBERNETES, COMMON FAILURE POINTS
AND HOW TO INVESTIGATE DNS ISSUES

1 CoreDNS Request Path



2 Service Discovery Flow

- Pods query Kubernetes Services using DNS.
- Service name is resolved to ClusterIP.
- Enables communication using stable names (e.g. my-service.default.svc.cluster.local).
- Works across namespaces using fully qualified domain names (FQDN).

3. SEARCH DOMAINS

- Kubernetes adds default search domains to `/etc/resolv.conf`.
- If you query a short name, the resolver tries multiple domains in order.
- Default search domains:
 - default.svc.cluster.local
 - svc.cluster.local
 - cluster.local
- This can generate multiple DNS queries per lookup.
- Helps with service discovery but can increase query volume.

4. NDOTS

- ndots controls when the resolver treats a name as absolute vs relative.
- Default is ndots:5 in Kubernetes.
- If a name has fewer than 5 dots, the resolver treats it as relative and appends search domains.
- This can cause multiple queries and increase latency.
- For external domains, use a trailing dot (e.g. example.com.) to avoid search domain expansion.

5. DNS AMPLIFICATION

- Occurs when one client query generates multiple DNS queries due to search domains and ndots.
- Can lead to high query volume and load on CoreDNS.
- Common with short names and misconfigured applications.
- Can cause CoreDNS saturation and increased latency.
- Use FQDN or trailing dot for external names to reduce amplification.

6. RESOLVER BEHAVIOR

- The resolver in the container (usually glibc or musl) handles retries and timeouts.
- Queries are sent to the nameserver (CoreDNS ClusterIP).
- If no response, the resolver retries (based on timeout settings).
- Applications may cache DNS responses.
- Different languages and runtimes have different DNS caching behavior.
- Check `/etc/resolv.conf` and application settings when debugging.

7. COREDNS SATURATION

- High query volume can saturate CoreDNS.
- Symptoms:
 - High CPU and memory usage
 - Increased latency
 - Timeouts (SERVFAIL, i/o timeout)
 - Dropped queries
 - Pods unable to resolve services
- Common causes:
 - DNS amplification
 - Misbehaving applications
 - Large number of short-name queries
 - Insufficient replicas or resources
- Monitor CoreDNS metrics and logs.

8. INTERMITTENT DNS INCIDENT INVESTIGATION

- Check symptoms (timeouts, errors, affected services).
- Test DNS from a pod:


```
nslookup <service-name>
dig <service-name>
```
- Check CoreDNS pods: `kubectl get pods -n kube-system`
- Check CoreDNS logs: `kubectl logs -n kube-system -l k8s-app=kube-dns`
- Check CoreDNS metrics (CPU, memory, queries, errors).
- Look for DNS amplification (high query rate).
- Verify network connectivity to CoreDNS.
- Check upstream DNS if external domains fail.
- Review application resolver settings (ndots, caching).
- Identify root cause and apply fix (scale, config, optimize).

COMMON DNS ERRORS

- timeout: no response from DNS server
- SERVFAIL: server failed to process query
- NXDOMAIN: name does not exist
- no such host: application error
- i/o timeout: resolver could not reach server

SENIOR ENGINEER TIP

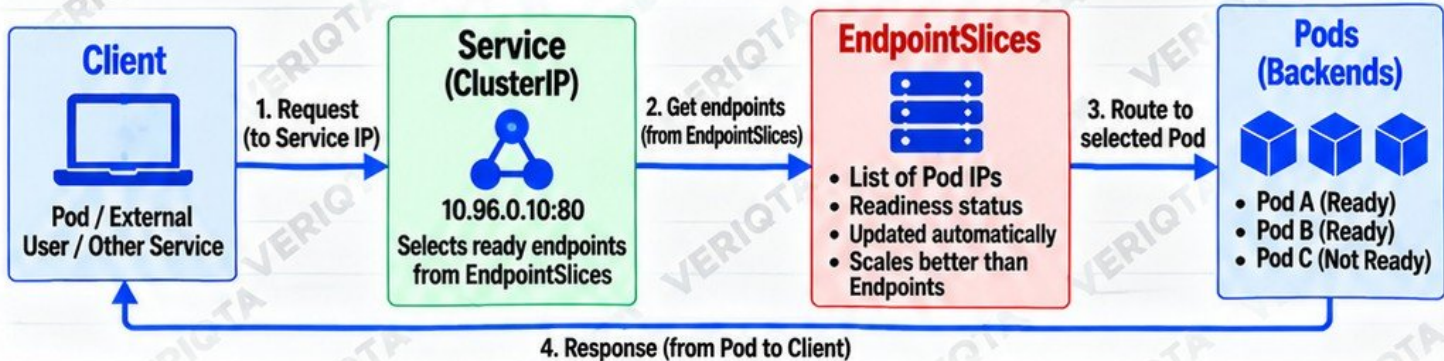
- Use FQDN for external domains (e.g. example.com.).
- Monitor CoreDNS metrics (query rate, latency, error rate).
- Set appropriate resource requests/limits.
- Use multiple replicas and PodDisruptionBudgets.
- Check both network and application configuration.

GOAL

Understand how DNS works in Kubernetes, identify the root cause of DNS failures, and apply the right fix to restore service connectivity.

Services, EndpointSlices and Traffic Routing

HOW KUBERNETES SERVICES WORK, HOW ENDPOINTS ARE SELECTED, AND HOW TRAFFIC IS ROUTED TO YOUR PODS



1. CLUSTERIP

- Default Service type inside the cluster.
- Provides a stable virtual IP (ClusterIP) and DNS name.
- Load balances traffic to healthy Pods using kube-proxy.
- Only accessible within the cluster (unless exposed via other Service types).
- Example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
```

2. ENDPOINTSlices

- Modern replacement for Endpoints.
- Stores a list of Pod IPs for a Service.
- Includes readiness, topology and additional metadata.
- Scales to thousands of endpoints.
- Automatically updated by Kubernetes when Pods are added, removed or become unready.

```
kubectl get endpointslices
kubernetes.io/service-name=my-service
```

4. KUBE-PROXY BEHAVIOR

- Runs on every node.
- Watches Services and EndpointSlices.
- Programs network rules using iptables, IPVS or eBPF (depending on mode).
- Forwards traffic from ClusterIP to a selected backend Pod.
- Only routes to ready endpoints.

3. READINESS AND TRAFFIC ELIGIBILITY

- Only Pods that are Ready receive traffic.
- Readiness probes determine if a Pod is Ready.
- If a Pod becomes unready, it is removed from EndpointSlices.
- This prevents traffic from going to unhealthy Pods.
- Check readiness status:

```
kubectl get pods -o wide
kubectl describe pod <pod-name>
```

5. LOAD DISTRIBUTION

- kube-proxy distributes traffic across all ready endpoints.
- In iptables mode: uses probabilistic load balancing.
- In IPVS mode: uses more advanced algorithms (e.g. round-robin, least-connections).
- In eBPF-based CNIs: load balancing may be handled by the CNI instead.
- Traffic is generally evenly distributed but not guaranteed to be perfectly balanced.

6. STALE ENDPOINTS

- Can happen when Pods terminate but are still listed temporarily.
- Usually self-heals when controllers update EndpointSlices.
- May occur during rapid scaling, network delays or control-plane issues.
- Check for stale endpoints:

```
kubectl get endpointslices
kubectl describe endpointslice <name>
```

7. TRAFFIC BLACK HOLES

- Occurs when traffic is sent to a Pod that cannot actually receive it.
- Causes include:
 - Pod not Ready but still in endpoints (temporary)
 - Network policy blocking traffic
 - Pod listening on wrong port
 - Application not responding
 - CNI or kube-proxy misconfiguration
 - Service selector not matching the right Pods
- Check connectivity from a test Pod:

```
kubectl run -it --rm test --image=curlimages/curl --
curl -v http://my-service
```

SENIOR ENGINEER TIP

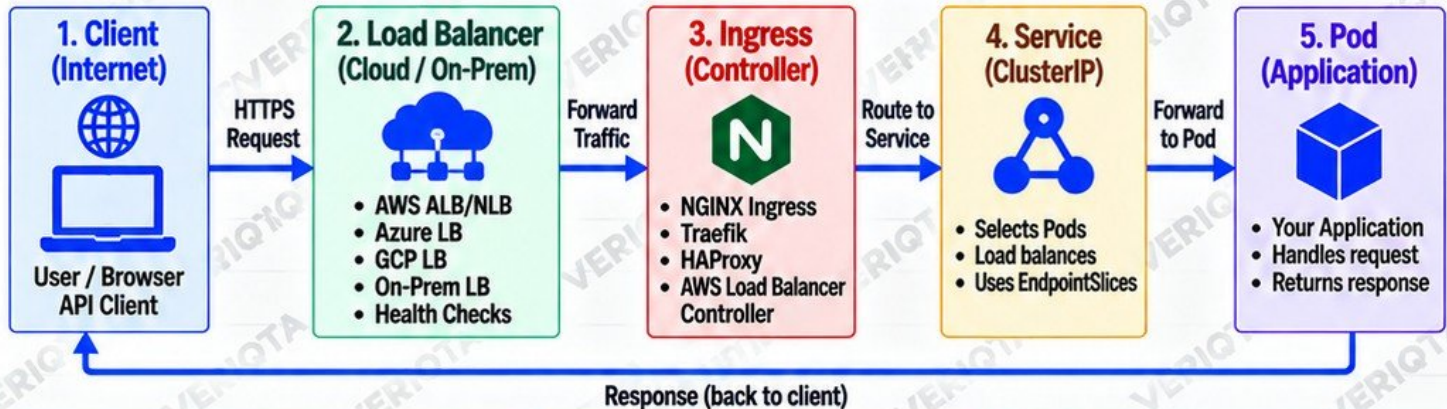
- Always check EndpointSlices when troubleshooting Service issues.
- Verify Pod readiness and actual application health (not just container running).
- Check kube-proxy mode and rules on the node.
- Use network policies and monitoring to detect routing problems early.

GOAL

Understand how Services and EndpointSlices work, how traffic is routed, and how to identify and fix routing issues in production.

Ingress, Load Balancers and the External Request Path

HOW EXTERNAL REQUESTS REACH YOUR PODS, KEY COMPONENTS, COMMON ISSUES AND HOW TO TROUBLESHOOT



1. CLIENT → LB → INGRESS → SERVICE → POD

- Client sends an HTTPS/HTTP request to your domain (e.g. app.example.com).
- Request reaches the Load Balancer (cloud or on-prem).
- LB forwards traffic to the Ingress controller.
- Ingress matches host/path rules and routes to the correct Service.
- Service load balances to a healthy Pod (using EndpointSlices).
- Response follows the reverse path back to the client.

2. TLS TERMINATION

- TLS can be terminated at:
 - Load Balancer (e.g. AWS ALB)
 - Ingress controller (e.g. NGINX)
 - Or both (SSL passthrough)
- After termination, traffic to the Service is usually HTTP (inside the cluster).
- Use valid certificates (Let's Encrypt, ACM, etc).
- Ensure correct certificate, host and SNI configuration.

3. HEALTH CHECKS

- Load balancer health checks ensure the Ingress is healthy.
- Ingress may also have its own health endpoints (e.g. /healthz).
- If health checks fail, the LB stops sending traffic to unhealthy nodes.
- Common issues:
 - Wrong path or port
 - Ingress not ready
 - Firewall or security group blocking

4. SOURCE IP BEHAVIOR

- By default, the source IP may be lost due to NAT (at the load balancer).
- Use these to preserve the real client IP:
 - AWS: enable X-Forwarded-For (default)
 - NGINX Ingress: use real_ip_header
 - cloud provider specific annotations
- Check client IP in your application logs:
 - X-Forwarded-For header
 - or PROXY protocol (for TCP)

5. TIMEOUTS

- Multiple timeout layers can cause failures:
 - Client timeout (browser, API client)
 - Load balancer idle timeout
 - Ingress proxy timeouts (e.g. NGINX)
 - Service/Pod application timeouts
- If a request takes longer than any timeout value, it may be terminated.
- Tune timeouts based on your application needs (e.g. long-running requests).

6. CONNECTION REUSE

- Load balancers and Ingress controllers reuse connections (keep-alive) for better performance.
- HTTP/1.1 keep-alive and HTTP/2 allow multiple requests over the same connection.
- Misconfigured keep-alive or very short idle timeouts can cause unnecessary connection churn and increased latency.
- Some applications may not handle connection reuse well, leading to intermittent errors.

7. 502, 503 AND 504 DIAGNOSIS

Error Code	Meaning	Common Causes	What to Check
502 Bad Gateway	Ingress/LB received an invalid response from upstream	Pod not ready App crash Wrong port	Pod logs Service/Endpoints Ingress config
503 Service Unavailable	No healthy upstream available	No ready Pods Failed health checks Scaling issues	Endpoints Pod readiness LB health checks
504 Gateway Timeout	Upstream did not respond in time	Application slow Timeout too short Network issues	Ingress/LB timeouts App performance Network connectivity

COMMON TROUBLESHOOTING COMMANDS

```

# Check ingress
kubectl get ingress
kubectl describe ingress <name>

# Check service and endpoints
kubectl get svc
kubectl get endpointslices

# Check pod logs
kubectl logs <pod-name>

# Test from inside cluster
curl -v http://<service-name>.<namespace>.svc.cluster.local
  
```

SENIOR ENGINEER TIP

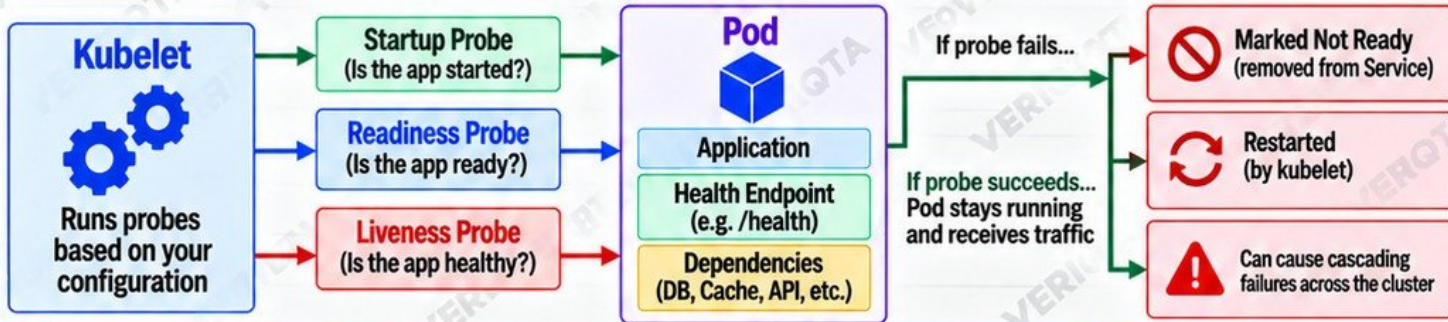
- Trace the full request path end-to-end.
- Check health, logs and metrics at every layer (LB, Ingress, Service, Pod).
- Use distributed tracing (e.g. OpenTelemetry) for complex issues.
- Always verify DNS, TLS, timeouts and security groups.

GOAL

Understand the complete external request path, quickly diagnose 502, 503 and 504 errors, and ensure reliable and secure access to your applications.

Probes Can Take Production Down

HOW KUBERNETES PROBES WORK, WHY THEY FAIL,
AND HOW THEY CAN BRING DOWN YOUR APPLICATION IN PRODUCTION



1. STARTUP PROBES

- Used to check if the application has started.
- Disables liveness and readiness probes until it succeeds.
- Useful for slow-starting applications (e.g. JVM, large caches).
- Prevents premature restarts during initialization.

Example:

```

startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
  
```

2. READINESS PROBES

- Checks if the application is ready to serve traffic.
- If it fails, the Pod is marked Not Ready and removed from Service endpoints.
- Helps prevent sending traffic to a Pod that cannot handle requests.
- Should check real application readiness (including dependencies when needed).

Example:

```

readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  
```

3. LIVENESS PROBES

- Checks if the application is still healthy.
- If it fails, kubelet restarts the container.
- Should only check if the app is truly stuck (not for dependency failures).
- Misconfigured liveness probes can cause unnecessary restarts and outages.

Example:

```

livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
  
```

4. PROBE TIMING

- Key parameters:
 - initialDelaySeconds
 - periodSeconds
 - timeoutSeconds
 - failureThreshold
 - successThreshold (for readiness)
- Set realistic values based on your application startup time.
- Avoid aggressive timeouts that cause false failures.
- Consider network latency and resource constraints.

5. DEPENDENCY-SENSITIVE PROBES

- Decide what to check in your health endpoint.
- Avoid failing probes due to temporary external dependency issues (e.g. database, third-party API).
- Use separate endpoints for liveness (simple) and readiness (deeper checks).
- Consider graceful degradation when dependencies are unavailable.

6. RESTART LOOPS

- Happens when liveness probe keeps failing.
- Container is killed and restarted repeatedly.
- Leads to CrashLoopBackOff.
- Can cause service downtime if all replicas keep restarting.
- Check logs, probe configuration, and application start time.

7. CASCADING PROBE FAILURES

- One failing dependency can cause multiple Pods to fail probes.
- Leads to mass restarts and loss of capacity.
- Can trigger a chain reaction across services.
- Example: database slowdown → app readiness fails → pods removed → increased load on remaining pods → more failures.
- Use timeouts, retries and circuit breakers to reduce blast radius.

8. SAFER HEALTH-CHECK DESIGN

- Use startup probes for slow apps.
- Keep liveness probes simple (e.g. /healthz).
- Use readiness probes for real readiness checks.
- Set conservative timing values.
- Avoid dependency-heavy liveness checks.
- Test probe behavior in staging.
- Monitor probe failures (metrics and alerts).
- Design for resilience, not perfection.

COMMON MISTAKES

- Using liveness probes for dependency checks.
- Setting timeouts too low.
- No startup probe for slow applications.
- Ignoring probe failures in monitoring.
- Not testing probe behavior under load.

SENIOR ENGINEER TIP

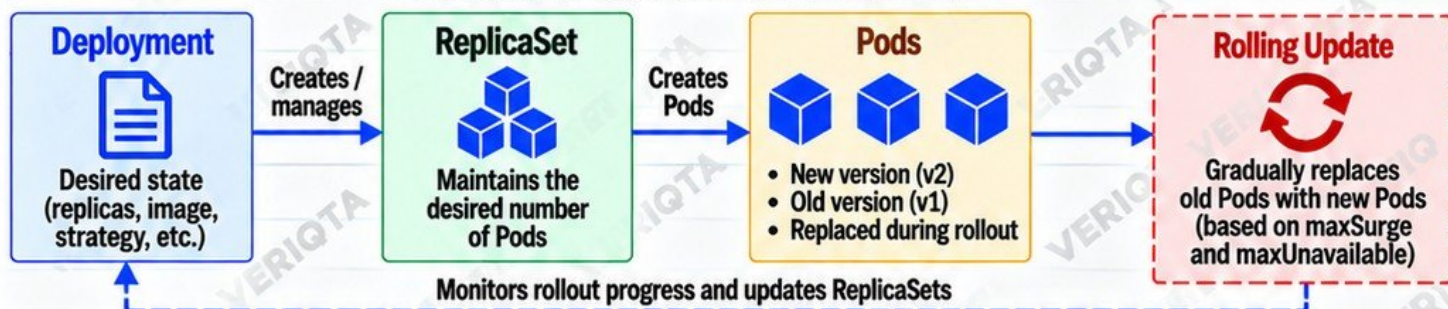
A bad probe can take a healthy application down. Design probes based on real application behavior, test failure scenarios, and always monitor probe metrics in production.

GOAL

Design and configure Kubernetes probes correctly, avoid common pitfalls, and prevent probe-related outages in production.

Deployments, Rollouts and Failed Releases

HOW KUBERNETES DEPLOYMENTS WORK, KEY CONCEPTS, COMMON FAILURE SCENARIOS AND HOW TO TROUBLESHOOT AND RECOVER



1. REPLICASETS

- A ReplicaSet ensures a specified number of identical Pods are running.
- Deployments create and manage ReplicaSets automatically.
- During a rollout, a new ReplicaSet is created for the new version, and the old ReplicaSet is scaled down.
- You usually do not manage ReplicaSets directly (unless troubleshooting).

```
kubectl get rs
kubectl describe rs <name>
kubectl get pods -l app=<app-name>
```

4. MAXUNAVAILABLE

- Controls how many Pods can be unavailable during the update.
- Can be a number or a percentage.
- Example: maxUnavailable: 0 (no downtime) or maxUnavailable: 25%.
- Set to 0 for critical applications to maintain full availability.

```
spec:
  strategy:
    rollingUpdate:
      maxUnavailable: 0 # or 25%
```

7. FAILED ROLLOUT DIAGNOSIS

- Check rollout status and events.
- Identify why new Pods are not becoming Ready.
- Common causes:
 - image pull errors
 - failing readiness probes
 - insufficient resources
 - misconfiguration (env vars, secrets, config)
 - application crashes

```
kubectl rollout status deployment/<name>
kubectl describe deployment <name>
kubectl get events --sort-by=.lastTimestamp
```

2. ROLLINGUPDATE MECHANICS

- Default strategy for Deployments.
- Creates new Pods with the new version.
- Gradually shifts traffic to new Pods as they become Ready.
- Terminates old Pods based on maxSurge and maxUnavailable.
- Ensures zero or minimal downtime when configured correctly.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
```

5. READINESS GATES

- Allow you to define additional conditions that a Pod must meet before it is considered Ready.
- Useful when using external systems (e.g. load balancers, service meshes).
- Prevents traffic from being sent to a Pod until all conditions are satisfied.

```
spec:
  template:
    spec:
      readinessGates:
        - conditionType: example.com/ready
```

8. ROLLBACK VS ROLL-FORWARD

Rollback

- Revert to the previous working version.
- Fastest way to restore service.
- Useful for critical production issues.

```
kubectl rollout undo deployment/<name>
kubectl rollout history deployment/<name>
```

Roll-Forward

- Fix the issue and release a new version.
- Recommended for long-term stability.
- Ensures you don't stay on an old version.

```
kubectl set image deployment/<name> <container>=<new-image>
kubectl rollout status deployment/<name>
```

3. MAXSURGE

- Controls how many extra Pods can be created above the desired replica count during an update.
- Can be a number or a percentage.
- Example: maxSurge: 1 (allows 1 extra Pod) or maxSurge: 25%.
- Helps ensure new Pods are ready before old Pods are removed.

```
spec:
  strategy:
    rollingUpdate:
      maxSurge: 1 # or 25%
```

6. PROGRESS DEADLINES

- Specifies the maximum time allowed for a rollout to make progress.
- If the new Pods do not become Ready within this time, the Deployment is marked as ProgressDeadlineExceeded.
- Helps detect stuck or failed rollouts early.

```
spec:
  progressDeadlineSeconds: 600
```



SENIOR ENGINEER TIP

- Always monitor rollouts in real time.
- Set appropriate maxSurge and maxUnavailable for your workload.
- Use meaningful readiness probes.
- Set a progress deadline.
- Automate rollback for critical services.
- Prefer roll-forward after root cause analysis and a proper fix.

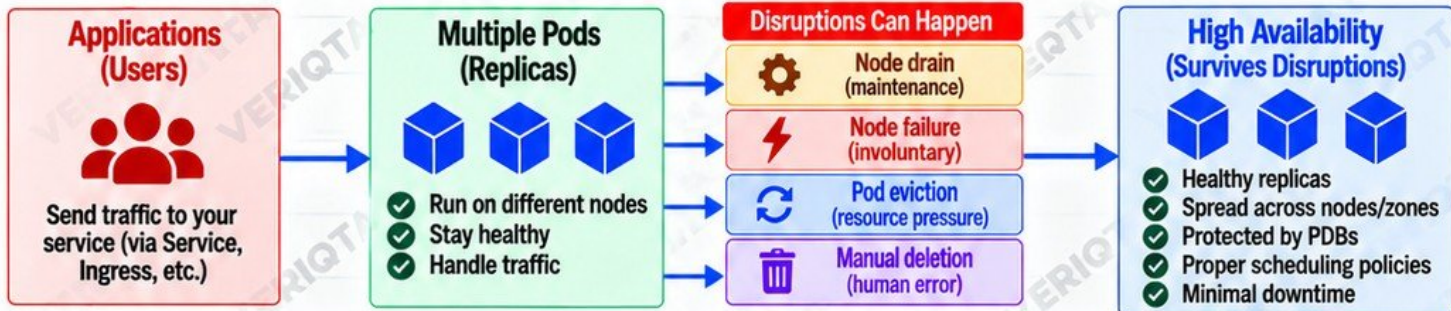


GOAL

Understand how Deployments and rollouts work, configure them correctly, quickly diagnose failed releases, and recover without prolonged downtime.

Pod Disruption and High Availability

HOW POD DISRUPTIONS HAPPEN, HOW TO MINIMIZE THEM,
AND WHY REPLICAS ALONE DO NOT GUARANTEE HIGH AVAILABILITY



1. VOLUNTARY VS INVOLUNTARY DISRUPTION

- Voluntary disruption: planned actions (e.g. node drain, rolling update, manual deletion).
- Involuntary disruption: unexpected events (e.g. node failure, power loss, network issue, OOM eviction).
- Both can make Pods unavailable and affect your application.

2. PODDISRUPTIONBUDGETS (PDBs)

- PDBs limit the number of Pods that can be voluntarily disrupted at the same time.
- Helps maintain application availability during maintenance events.
- Defined using minAvailable or maxUnavailable.
- Does not protect against involuntary disruptions (e.g. node failure).

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: my-app
```

3. NODE DRAINS

- Used for planned maintenance (e.g. OS upgrade, node replacement).
- Kubernetes evicts Pods gracefully using the eviction API.
- Respects PDBs (will not evict more Pods than allowed).
- Use cordon to mark the node unschedulable, then drain.

```
kubectl cordon <node-name>
kubectl drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data
```

4. REPLICA PLACEMENT

- Spread replicas across multiple nodes to avoid single node failure.
- Use podAntiAffinity to keep replicas from running on the same node.
- Ensure you have enough nodes to run replicas.

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: my-app
          topologyKey: kubernetes.io/hostname
```

5. ZONE-AWARE SCHEDULING

- Distribute replicas across multiple availability zones (AZs).
- Use topologySpreadConstraints or node affinity with zone labels.
- Protects against zone-level failures.

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app: my-app
```

6. MAINTENANCE EVENTS

- Common maintenance events:
 - Kubernetes upgrades
 - Node OS upgrades
 - Cloud provider maintenance
 - Cluster autoscaler actions
 - Security patching
 - Hardware replacement
- Use PDBs, proper replica placement and monitoring to minimize impact.

7. WHY REPLICAS DO NOT AUTOMATICALLY EQUAL HA

- Replicas can still be on the same node or same zone.
- A single node or zone failure can take all replicas down.
- Misconfigured probes, resource limits or dependencies can make all replicas unhealthy.
- HA requires: multiple nodes, multiple zones, correct scheduling, PDBs, healthy dependencies and monitoring.

8. PRACTICAL EXAMPLE

3 replicas across 3 zones with a PDB

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: my-app
```

COMMON MISTAKES

- Relying only on replicas for HA.
- Not using PDBs.
- Running all replicas on the same node/zone.
- Ignoring maintenance events.
- Not testing failure scenarios.

SENIOR ENGINEER TIP

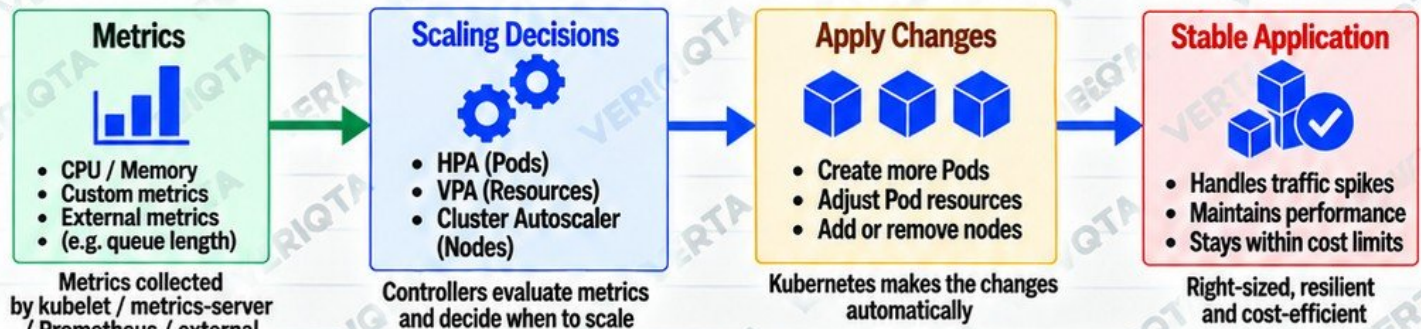
- Design for failure, not perfection.
- Use PDBs, zone-aware scheduling and anti-affinity.
- Test disruptions regularly (game days).
- Monitor Pod availability and alert on replica drops.
- Ensure dependencies (DB, cache, APIs) are also HA.

GOAL

Build applications that can survive planned and unplanned disruptions with minimal downtime.

Autoscaling Without Creating Another Incident

HOW HPA, VPA AND NODE AUTOSCALING WORK, COMMON PITFALLS, AND HOW TO SCALE SAFELY, EFFICIENTLY AND COST-EFFECTIVELY



1. HORIZONTAL POD AUTOSCALER (HPA)

- Scales the number of Pods based on metrics (e.g. CPU, memory, custom metrics).
- Uses a target value (e.g. 70% CPU).
- Increases or decreases replicas automatically.
- Works best with stateless applications.
- Relies on metrics from metrics-server or Prometheus (via custom.metrics.k8s.io).

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  
```

2. VERTICAL POD AUTOSCALER (VPA)

- Automatically adjusts CPU and memory requests/limits for Pods.
- Recommends or applies new resource values based on actual usage.
- Helps prevent OOM kills and right-size resources.
- Does not scale the number of Pods (works with replicas).
- Best used in "recommendation" mode first, then carefully in "auto" mode.

```

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
    - containerName: my-app
      minAllowed:
        cpu: "100m"
        memory: "256Mi"
      maxAllowed:
        cpu: "25%"
        memory: "4Gi"
  
```

3. NODE AUTOSCALING

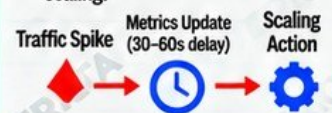
- Cluster Autoscaler automatically adds or removes nodes based on pending Pods.
- Works with cloud providers (ASA, GKE, EKS) or cluster autoscaler for self-managed clusters.
- Adds nodes when there are unschedulable Pods.
- Removes nodes when they are no longer needed.
- Considers resource requests (not actual usage).



Remember:
If your Pods have low or no requests, Cluster Autoscaler may not scale correctly.

4. METRICS LAG

- Metrics are not real-time.
- There is a delay between actual load and scaling action.
- During sudden spikes, it may take 30–60 seconds (or more) to react.
- This can cause temporary high latency or failed requests.
- Use appropriate metrics (e.g. custom metrics for queues) for faster and more accurate scaling.



Tip:
Use predictive metrics (e.g. queue length) instead of only CPU for faster response.

5. REQUESTS AND SCALING CALCULATIONS

- HPA uses resource requests to calculate utilization (not actual usage).
- If requests are set too high, scaling may not trigger.
- If requests are too low, scaling may happen too late.
- Set realistic requests based on real usage data.

CPU Utilization (%) =
(Actual CPU Usage / CPU Request) × 100

6. COLD STARTS

- New Pods take time to start (image pull, initialization, app startup).
- During cold starts, traffic may fail if scaling is too slow.
- Use faster container images, readiness probes and pre-warming strategies.
- Consider minimum replicas to handle expected traffic.

Mitigation:

- Use smaller images
- Keep a baseline number of replicas
- Optimize startup time
- Use startup probes

7. SCALING OSCILLATION

- Happens when the number of replicas too frequently goes up and down.
- Caused by short metrics windows, sensitive thresholds or variable traffic.
- Leads to instability, higher costs and unnecessary restarts.
- Use stabilization windows and appropriate thresholds.

Example (HPA stabilization):

```

behavior:
  scaleUp:
    stabilizationWindowSeconds: 60
  scaleDown:
    stabilizationWindowSeconds: 300
  
```

8. CAPACITY BOUNDARIES AND COST VS RESILIENCE

- Set minimum and maximum replicas based on business needs.
- Understand cluster and cloud provider limits (quotas, node groups, zones).
- More replicas improve resilience but increase cost.
- Find the right balance for your workload (SLO, budget, risk tolerance).

Key Considerations:

- Set resource quotas and limits
- Plan for peak traffic
- Use multi-AZ/node pools
- Balance cost vs reliability

COMMON MISTAKES

- No or unrealistic resource requests.
- Relying only on CPU metrics.
- Setting thresholds too low (causes oscillation).
- Ignoring cold start time.
- Not testing scaling under real load.
- Assuming autoscaling solves all availability issues.

SENIOR ENGINEER TIP

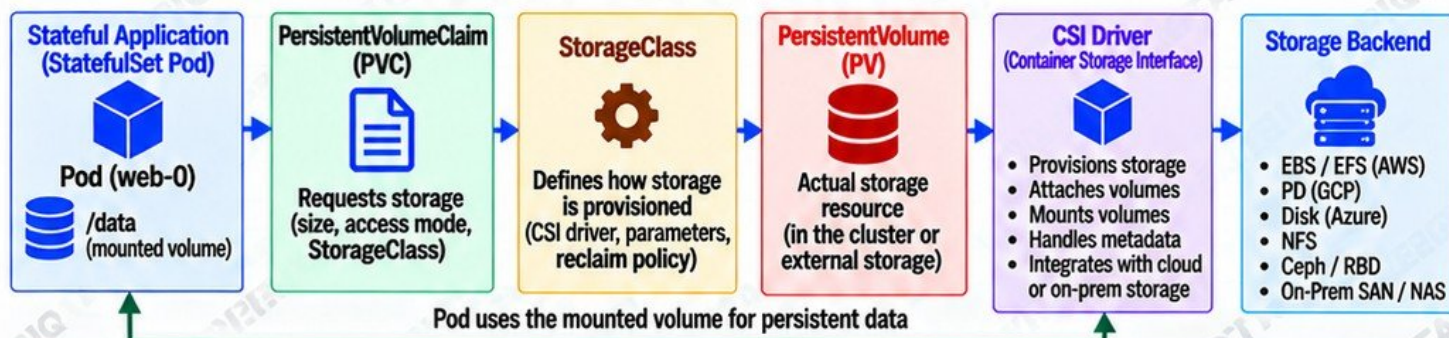
- Autoscaling is not a magic fix.
- Use the right metrics, set realistic thresholds, and test thoroughly.
- Combine HPA, VPA and Cluster Autoscaler wisely.
- Always monitor scaling events and application performance.
- Design for both resilience and cost efficiency.

GOAL

Scale your applications intelligently, avoid creating new incidents, and maintain a balance between performance, cost and reliability.

Storage Failures in Stateful Kubernetes

HOW KUBERNETES STORAGE WORKS, COMMON FAILURE SCENARIOS, AND HOW TO TROUBLESHOOT AND RECOVER



1. PV/PVC LIFECYCLE

- PVC is created by a user (or StatefulSet).
- PVC requests storage (size, access mode, StorageClass).
- StorageClass provisions a PV (dynamically or statically).
- PV is bound to the PVC.
- Pod mounts the volume and uses it.
- When PVC is deleted, PV is either retained or deleted (reclaim policy).

Check status:

```
kubectl get pvc
kubectl get pv
kubectl describe pvc <name>
kubectl describe pv <name>
```

2. STORAGECLASS

- Defines the provisioner (CSI driver).
- Sets parameters (disk type, IOPS, etc.).
- Controls reclaim policy (Retain, Delete).
- Can be default or custom.
- Different StorageClasses for different performance and availability needs.

Example:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  fsType: ext4
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
```

3. CSI (CONTAINER STORAGE INTERFACE)

- Standard interface for storage plugins.
- Works with cloud and on-prem storage.
- Handles provisioning, attaching, mounting and health checks.
- Examples: aws-ebs-csi, azure-disk-csi, gcp-pd-csi, nfs-csi, ceph-csi.
- CSI controllers run in the cluster (provisioner, attacher, resizer, etc.).

Check CSI components:

```
kubectl get pods -n kube-system |
grep csi
kubectl get csidrivers
kubectl get csinodes
```

4. ATTACH AND MOUNT

- Volume is attached to a node by the CSI driver.
- Kubelet mounts the volume into the Pod at the specified path.
- Attach happens at the node level (for block storage).
- Mount happens inside the Pod.
- Failures can occur during attach or mount (e.g. wrong node, permissions, driver issues).

Check mount status:

```
kubectl describe pod <pod-name>
kubectl get events --sort-by=.lastTimestamp
journalctl -u kubelet | grep -i mount
```

5. ACCESS MODES

- ReadWriteOnce (RWO): one node can mount read-write.
- ReadOnlyMany (ROX): many nodes can mount read-only.
- ReadWriteMany (RWX): many nodes can mount read-write (e.g. NFS, EFS).
- Choose the right mode for your application and storage backend.

Example in PVC:

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi
```

6. ZONE CONSTRAINTS

- Many cloud volumes are tied to a specific availability zone.
- Pods must be scheduled to nodes in the same zone as the volume.
- Use StorageClass with volumeBindingMode: WaitForFirstConsumer.
- Use topologySpreadConstraints or node affinity for StatefulSets.
- Cross-zone attachment is not supported for most block storage.

Check PV zone:

```
kubectl get pv <name> -o yaml | grep topology
kubectl get nodes -o wide
```

7. STATEFULSET IDENTITY

- Each Pod in a StatefulSet has a stable identity (e.g. web-0, web-1).
- Each Pod gets its own PVC, which keeps the same data even after restart.
- Pods keep the same network identity (stable DNS name).
- Deleting and recreating a Pod reuses the same PVC.
- Scaling up creates new PVCs for new replicas.

Check StatefulSet and PVCs:

```
kubectl get sts
kubectl get pods -l app=<name>
kubectl get pvc -l app=<name>
```

8. STUCK VOLUMES

- PVC stuck in Pending (no matching PV).
- PV stuck in Terminating.
- Volume stuck in "Attachment" or "Detaching" state.
- Pod stuck in ContainerCreating (mount failure).
- Causes: wrong zone, resource limits, CSI driver issues, deleted nodes, finalizers, cloud provider issues.

Troubleshoot:

```
kubectl describe pvc <name>
kubectl describe pv <name>
kubectl get events
kubectl get volumeattachments
kubectl get pods -o wide
```

9. STORAGE LATENCY AND RECOVERY

- High I/O latency can cause application slowness, timeouts and failures.
- Monitor storage metrics (IOPS, throughput, latency).
- Check if volume is throttled by cloud provider.
- For recovery: identify cause, fix underlying storage issue, restart affected Pods if needed.
- Consider using faster storage classes or caching.

COMMON MISTAKES

- Using the wrong StorageClass or access mode.
- Ignoring zone constraints.
- Deleting PVCs accidentally.
- Not monitoring storage performance.
- Assuming StatefulSets automatically provide high availability.
- Not testing storage failure scenarios.

SENIOR ENGINEER TIP

- Always use proper StorageClass settings.
- Monitor volume health, latency and IOPS.
- Use zone-aware scheduling.
- Test failure and recovery procedures.
- Set reclaimPolicy based on your data retention needs.
- Document storage dependencies for your applications.

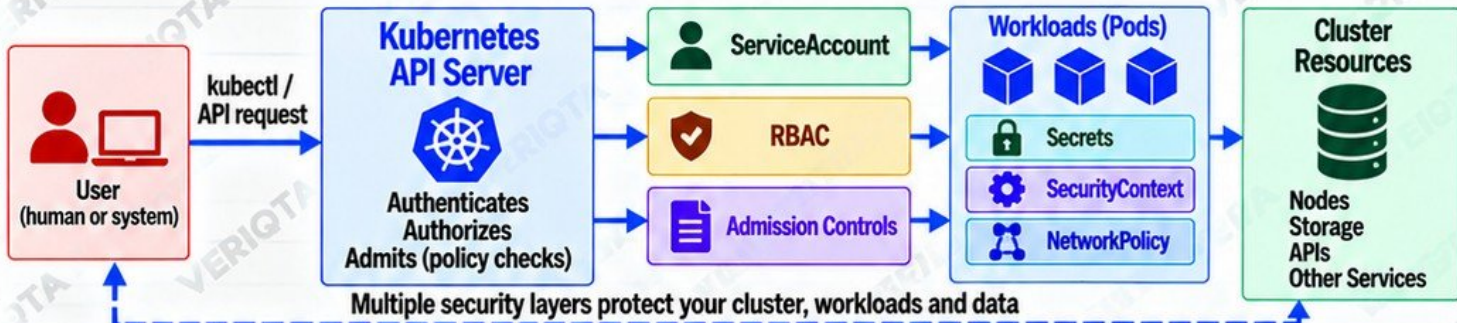
GOAL

Keep your stateful applications running by understanding Kubernetes storage, preventing common failures, and knowing how to recover quickly.

Kubernetes Security Boundaries

HOW KUBERNETES SECURITY WORKS, CORE BOUNDARIES, COMMON RISKS,
AND HOW TO BUILD A MORE SECURE CLUSTER

LEARN
BUILD
OPERATE
SCALE



1. SERVICEACCOUNTS

- Used by Pods to authenticate to the API server.
- Each namespace has a default ServiceAccount.
- Use dedicated ServiceAccounts per application.
- Avoid using the default ServiceAccount.
- Combine with RBAC for least privilege.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: my-app
```

2. RBAC (ROLE-BASED ACCESS)

- Controls what users, groups and ServiceAccounts can do.
- Use Roles (namespace) or ClusterRoles (cluster-wide).
- Follow the principle of least privilege.
- Regularly review and clean up permissions.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: my-app
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

3. WORKLOAD IDENTITY

- Maps Kubernetes identities to cloud identities (e.g. IAM).
- Avoid long-lived static credentials in Pods.
- Use IRSA (AWS), Workload Identity (GCP), or Azure Workload Identity.
- Gives temporary, least-privilege access to cloud resources.

Example (AWS IRSA):

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  annotations:
    eks.amazonaws.com/role-arn:
      arn:aws:iam::123456789012:role/app-role
```

4. SECRETS

- Store sensitive data (passwords, API keys, tokens).
- Use Kubernetes Secrets or external secret managers (e.g. AWS Secrets Manager, HashiCorp Vault).
- Do not hardcode secrets in images or manifests.
- Use encryption at rest and limit access via RBAC.

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: my-app
type: Opaque
data:
  username: <base64>
  password: <base64>
```

5. SECURITYCONTEXT

- Controls user, groups, privileges and filesystem access for containers.
- Run as non-root whenever possible.
- Drop unnecessary capabilities.
- Use read-only root filesystem.
- Helps reduce the impact of container compromise.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1800
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
```

6. POD SECURITY

- Enforce security standards for Pods (e.g. restricted, baseline).
- Use Pod Security Standards (PSS) or third-party tools.
- Prevent privileged containers and dangerous configurations.
- Apply at namespace level.

```
# Namespace with restricted profile
apiVersion: v1
kind: Namespace
metadata:
  name: my-app
  labels:
    pod-security.kubernetes.io/enforce: restricted
```

7. ADMISSION CONTROLS

- Validate and mutate resource requests before they are created.
- Use built-in controllers or tools like OPA/Gatekeeper, Kyverno.
- Enforce security policies (e.g. no privileged Pods, required labels, approved registries).
- Helps prevent misconfigurations and risky deployments.

```
# Example using Kyverno (simplified)
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-privileged
spec:
  validationFailureAction: enforce
  rules:
  - name: check-privileged
    match:
      resources:
        kinds: ["Pod"]
```

8. NETWORKPOLICY

- Controls network traffic between Pods, namespaces and external endpoints.
- Default allow (no policy) means all traffic is allowed.
- Use NetworkPolicies to restrict traffic (default deny + allow rules).
- Helps contain threats and reduce lateral movement.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-frontend
  namespace: my-app
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
```

9. CONTAINER CAPABILITIES

- Linux capabilities give granular permissions (e.g. NET_ADMIN).
- Drop all unnecessary capabilities.
- Add only what is required.
- Avoid privileged mode.
- Helps limit what a compromised container can do.

```
securityContext:
  capabilities:
    drop:
      - ALL
    add:
      - NET_BIND_SERVICE
  privileged: false
```

10. BLAST-RADIUS REDUCTION

- Limit the impact of a security incident.
- Use namespaces, RBAC, quotas and NetworkPolicies.
- Isolate workloads and data.
- Apply least privilege everywhere.
- Monitor, audit and detect suspicious activity.
- Combine multiple controls (defense in depth).

Key Strategies:

- ✓ Namespace isolation
- ✓ Least privilege access
- ✓ Network segmentation
- ✓ Resource limits and quotas
- ✓ Regular security audits
- ✓ Monitoring and alerting
- ✓ Use immutable infrastructure

COMMON MISTAKES

- Using default ServiceAccount.
- Overly permissive RBAC rules.
- Hardcoding secrets in images.
- Running containers as root or privileged.
- No NetworkPolicies.
- Not enforcing security policies.
- Assuming one control is enough.

SENIOR ENGINEER TIP

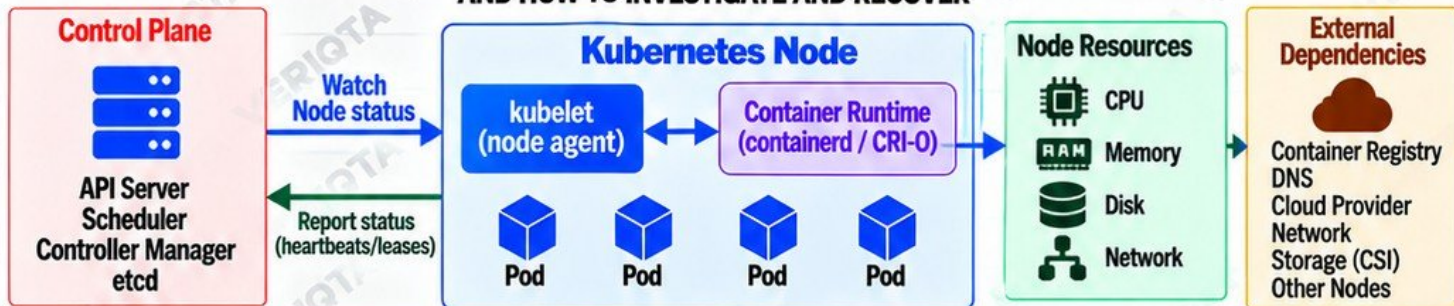
- ✓ Apply the principle of least privilege.
- ✓ Use multiple layers of security (defense in depth).
- ✓ Automate security checks in CI/CD.
- ✓ Regularly review permissions, secrets and configurations.
- ✓ Monitor and log for suspicious activity.
- ✓ Design for detection, containment and quick recovery.

GOAL

Build and operate Kubernetes clusters and applications with strong security boundaries, least privilege, and reduced blast radius.

Node Failure Anatomy

HOW A KUBERNETES NODE WORKS, WHAT HAPPENS WHEN IT FAILS,
AND HOW TO INVESTIGATE AND RECOVER



1. KUBELET

- Runs on every node.
- Registers the node with the API server.
- Manages Pods, containers, volumes, and node health.
- Reports node status and sends heartbeats (leases).
- If kubelet stops, the node will become NotReady.

Check kubelet status:

```
systemctl status kubelet
journalctl -u kubelet -n 100
kubectl get nodes
kubectl describe node <node-name>
```

2. CONTAINER RUNTIME

- Runs and manages containers (e.g. containerd, CRI-O).
- Pulls images, creates and starts containers.
- If the runtime is down, new Pods cannot start and existing Pods may fail.
- Kubelet communicates with the runtime via the CRI (Container Runtime Interface).

Check runtime status (containerd):

```
systemctl status containerd
crictl ps
crictl info
journalctl -u containerd -n 100
```

3. NODE CONDITIONS

- Show the current health of the node.
- Key conditions:
 - Ready
 - MemoryPressure
 - DiskPressure
 - PIDPressure
 - NetworkUnavailable
- Conditions are updated by kubelet and monitored by the control plane.
- Used by the scheduler and eviction logic.

View node conditions:

```
kubectl get nodes
kubectl describe node <node-name>
```

4. HEARTBEATS AND LEASES

- Kubelet sends regular heartbeats using Node Leases (coordination.k8s.io).
- If the control plane does not receive heartbeats, the node is marked NotReady.
- Default lease duration is 40 seconds (node-monitor-grace-period).
- Network issues, high load or kubelet crashes can cause missed heartbeats.

Check node lease:

```
kubectl get lease -n kube-node-lease
kubectl describe lease <node-name>
kubectl get node <node-name> -o yaml
```

5. NOTREADY BEHAVIOR

- When heartbeats are missed, the node is marked NotReady.
- Scheduler stops placing new Pods on the node.
- Existing Pods keep running for a while.
- After the toleration period, Pods may be evicted (if they do not tolerate the NotReady taint).
- When the node recovers, it becomes Ready and can accept new Pods again.

Check node status and taints:

```
kubectl get nodes
kubectl describe node <node-name>
kubectl get pods -o wide
kubectl get pods --all-namespaces -o wide
```

6. DISKPRESSURE

- Triggered when the node's disk space is low (e.g. /var/lib/kubelet, (containerd, or node filesystem).
- Can cause Pod eviction and prevent new Pods from starting.
- Often caused by:
 - Large container logs
 - Image buildup
 - Full root or data partition

Check disk usage:

```
df -h
du -sh /var/lib/kubelet/*
du -sh /var/log/*
kubectl describe node <node-name>
```

7. MEMORYPRESSURE

- Triggered when the node is low on memory.
- Can cause eviction of Pods (starting with lowest priority).
- Often caused by:
 - Memory leaks
 - Large workloads
 - Insufficient node memory
 - High system usage

Check memory usage:

```
free -h
top
kubectl describe node <node-name>
kubectl get pods --sort-by=.spec.nodeName
```

8. PIDPRESSURE

- Triggered when the number of processes (PIDs) on the node is high.
- Can prevent new containers from starting and may cause eviction.
- Often caused by:
 - Too many containers
 - Runaway processes
 - Misconfigured applications

Check PID usage:

```
cat /proc/sys/kernel/pid_max
ps aux | wc -l
kubectl describe node <node-name>
kubectl get pods --all-namespaces -o wide
```

9. POD EVICTION AND RESCHEDULING

- Kubelet evicts Pods when node resources are low (Disk, Memory or PID pressure).
- Evicted Pods are marked as Failed (Evicted).
- Deployments/StatefulSets will create new Pods on healthy nodes (if replicas are available).
- Some Pods may not be rescheduled immediately due to resource constraints or node selectors.

Check evicted Pods:

```
kubectl get pods --all-namespaces | grep Evicted
kubectl describe pod <pod-name>
```

10. NODE-LEVEL EVIDENCE

- Check kubelet logs.
- Check container runtime logs.
- Check system logs (dmesg, syslog).
- Look for OOM kills, disk errors, network issues or hardware problems.
- Check cloud provider events (if using a managed cluster).

Common evidence commands:

```
journalctl -u kubelet -n 100
journalctl -u containerd -n 100
dmesg | tail -n 50
df -h
kubectl describe node <node-name>
```

COMMON MISTAKES

- Ignoring node conditions.
- Not monitoring disk, memory and PID usage.
- Leaving too many container logs.
- Using nodes with insufficient resources.
- Not setting resource requests and limits.
- Ignoring underlying infrastructure issues (hardware, network, cloud).
- Assuming Kubernetes will always recover automatically.

SENIOR ENGINEER TIP

- Monitor node health proactively (Prometheus, Grafana, alerting).
- Set realistic resource requests and limits.
- Use log rotation and image cleanup.
- Monitor node conditions and leases.
- Have runbooks for node failure scenarios.
- Test recovery procedures regularly.
- Use multiple nodes and zones for high availability.

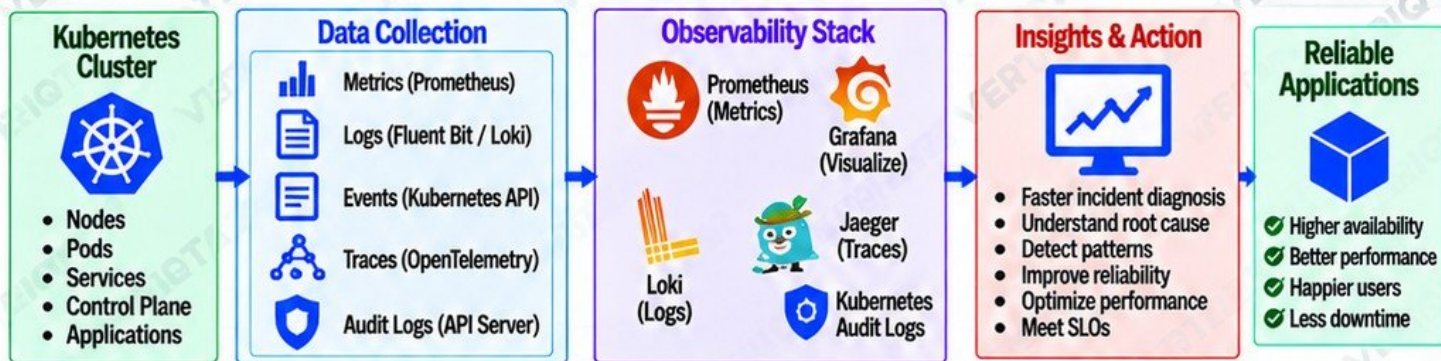
GOAL

Detect node issues early, minimize impact, and ensure fast recovery to keep your cluster healthy and resilient.

Kubernetes Observability That Actually Helps During Incidents

LEARN
BUILD
OPERATE
SCALE

TURN METRICS, LOGS, EVENTS AND TRACES INTO REAL INSIGHTS,
SO YOU CAN FIND AND FIX PROBLEMS FASTER



1. METRICS

- Time-series data about cluster and application health.
- Examples: CPU, memory, requests, latency, errors.
- Collected by Prometheus (often via kubelet, cadvisor, exporters).
- Used for alerting and SLOs.

Example queries (PromQL):

```
rate(http_requests_total[5m])
node_cpu_seconds_total
container_memory_working_set_bytes
```

2. LOGS

- Detailed records of what happened in containers and components.
- Include application logs, system logs and control-plane logs.
- Common stack: Fluent Bit + Loki (or EFK/ELK).
- Essential for debugging and root cause analysis.

Example (kubectl):

```
kubectl logs <pod-name>
kubectl logs -n kube-system
<component>
kubectl logs -f <pod> --previous
```

3. EVENTS

- Short, real-time records of important cluster actions.
- Show scheduling, image pulls, errors, warnings, etc.
- Useful for quick troubleshooting during incidents.
- Stored in etcd (with limited TTL).

Check events:

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod-name>
kubectl get events -n <namespace>
```

4. TRACES

- Show the path of a request across services (distributed tracing).
- Help identify where latency or errors occur.
- Common tools: OpenTelemetry, Jaeger, Tempo.
- Critical for microservices and complex architectures.

Example (OpenTelemetry):

Instrument your application with OpenTelemetry SDK and export traces to Jaeger or Tempo.

5. API AUDIT LOGS

- Record all requests to the Kubernetes API server.
- Help with security, compliance and incident investigation.
- Show who did what, when and from where.
- Enable for production clusters (with proper retention).

Example configuration:

```
--audit-log-path=/var/log/kubernetes/audit.log
--audit-policy-file=audit-policy.yaml
```

6. KUBE-STATE-METRICS

- Exposes metrics about Kubernetes objects (Pods, Deployments, etc.).
- Complements node and container metrics.
- Useful for monitoring the desired vs actual state.
- Common metrics: kube_pod_status, kube_deployment_status.

Example metrics:

```
kube_pod_status_phase
kube_deployment_status_replicas_available
kube_node_status_condition
```

7. CONTROL-PLANE SIGNALS

- Monitor key control-plane components: API server, etcd, scheduler, controller-manager.
- Check for latency, errors and resource usage.
- Control-plane issues can affect the entire cluster.
- Use metrics, logs and API health endpoints.

Check control-plane health:

```
kubectl get componentstatuses
kubectl get --raw=/readyz
kubectl get --raw=/livez
kubectl logs -n kube-system <component>
```

8. CARDINALITY

- High cardinality = too many unique metric labels (e.g. pod names, user IDs).
- Can cause high memory usage and slow queries in Prometheus.
- Use labels wisely and avoid unbounded values.
- Keep metrics low-cardinality and meaningful.

Bad vs Good example:

- ✗ `http_requests_total{pod="pod-abc123"}`
- ✓ `http_requests_total{app="api"}`

9. SLO SIGNALS

- Use observability data to track Service Level Objectives (SLOs).
- Key signals: availability, latency, error rate, saturation.
- Helps you know if users are affected.
- Use tools like Prometheus + Grafana and alert on SLO breaches.

Example SLO (PromQL):

```
(sum(rate(http_requests_total{code!~"5.."}[5m]))-
/ sum(rate(http_requests_total[5m]))) * 100
```

10. WHAT TO INSPECT FIRST DURING AN OUTAGE

- Check user impact (SLOs, error rates, latency).
- Check recent events (`kubectl get events`).
- Check affected Pods (status, restarts, logs).
- Check node health (`kubectl get nodes, conditions`).
- Check control-plane status (API server, etcd, scheduler).
- Check metrics dashboards (Grafana).
- Check logs (application and system).
- Check traces (if latency or errors).
- Look for recent changes (deployments, config, scaling).
- Form a hypothesis and validate step by step.

A COMMON MISTAKES

- ✗ No observability setup before an incident.
- ✗ Relying only on logs.
- ✗ High cardinality metrics.
- ✗ Ignoring control-plane signals.
- ✗ Not setting SLOs or alerts.
- ✗ Not keeping audit logs.
- ✗ Looking at too much data instead of a focused approach.
- ✗ Missing resource limits and saturation signals.
- ✗ No runbook for incident debugging.

SENIOR ENGINEER TIP

During an incident, start with signals that tell you **USER IMPACT**, then drill down: Events → Pods → Nodes → Control Plane → Logs → Metrics → Traces.

Kubernetes Incident Investigation Playbook

A STRUCTURED APPROACH TO FIND, FIX AND PREVENT INCIDENTS
IN KUBERNETES ENVIRONMENTS



1. DEFINE BLAST RADIUS



- Understand the full impact.
- Identify what is affected (Pods, Services, namespaces, users, regions).
- Check if it is isolated or widespread.
- Determine business impact (e.g. downtime, errors, latency).

Useful commands:

```
kubectl get pods -A
kubectl get svc -A
kubectl get endpoints -A
```

2. ESTABLISH TIMELINE



- Find out when the issue started.
- Match events with deployments, scaling, node changes, or cloud events.
- Use logs, metrics and audit logs to build a timeline.
- Identify what happened before, during and after.

Check timeline:

```
kubectl get events -A --sort-by=.lastTimestamp
kubectl get deploy -A --sort-by=.metadata.creationTimestamp
kubectl get nodes -o wide
```

3. CHECK RECENT CHANGES



- Look for what changed recently.
- Check deployments, config maps, secrets, ingresses, node updates.
- Review Git commits, CI/CD pipelines and infrastructure changes.
- Do not assume, verify.

Find recent changes:

```
kubectl rollout history deploy <name>
kubectl describe deploy <name>
kubectl get configmap -A --sort-by=.metadata.creationTimestamp
kubectl get secrets -A --sort-by=.metadata.creationTimestamp
```

4. WORK FROM SYMPTOM TO DEPENDENCY



- Start with the visible symptom (e.g. 500 errors, timeouts).
- Trace dependencies (Service → Pod → DNS → Ingress → External service).
- Check if the issue is internal or external.
- Follow the request path step by step.

Map the path:

```
kubectl get svc,ep -n <namespace>
kubectl get ingress -A
kubectl get pod -o wide
kubectl exec -it <pod> -- curl <dependency>
```

5. EVENTS, METRICS, LOGS AND TRACES



- Use all available data sources.
- Events: quick overview of what Kubernetes saw.
- Metrics: see performance and resource trends.
- Logs: find errors and root causes.
- Traces: follow requests across services.

Useful commands:

```
kubectl get events -A
kubectl logs <pod> -n <namespace> --tail=100
kubectl top pod -A
kubectl get --raw /metrics
kubectl get --raw /api/v1/traces (if available)
```

6. TEST HYPOTHESES



- Form a hypothesis based on evidence.
- Test and validate (do not guess).
- Rule out possibilities systematically.
- Use targeted commands and controlled tests.
- Keep notes of what you test.

Example tests:

```
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- curl -v <url>
kubectl describe pod <pod>
kubectl get networkpolicy -A
```

7. MITIGATE SAFELY



- Take action to reduce impact.
- Use the safest option first (e.g. rollback, scale, restart).
- Avoid making things worse.
- Communicate with stakeholders.
- Document what you do.

Common mitigation actions:

```
kubectl rollout undo deploy <name>
kubectl scale deploy <name> --replicas=0
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets
```

8. VERIFY RECOVERY



- Confirm the issue is resolved.
- Check service health, metrics and logs.
- Validate end-to-end functionality.
- Ensure no new errors are appearing.
- Monitor for a period of time before closing the incident.

Verify:

```
kubectl get pods -A
kubectl get svc -A
kubectl get ingress -A
kubectl top nodes
kubectl get events -A --sort-by=.lastTimestamp
```

9. PRESERVE EVIDENCE



- Save logs, events and configurations.
- Capture timelines, screenshots and commands.
- This helps with root cause analysis, compliance and learning.
- Do not delete data too early.

Gather evidence:

```
kubectl get events -A > events.txt
kubectl logs <pod> -n <ns> > pod-logs.txt
kubectl describe <resource> > resource.txt
kubectl get all -A -o yaml > cluster-state.yaml
```

10. PREVENT RECURRENCE



- Find the root cause.
- Implement fixes and improvements.
- Add monitoring, alerts and runbooks.
- Update documentation and share lessons learned.
- Review processes to prevent similar incidents.

Examples:

```
Add health checks and alerts (Prometheus/Grafana)
Implement better testing in CI/CD
Use policy and admission controls
Improve capacity and resilience
Run incident post-mortem and action items
```

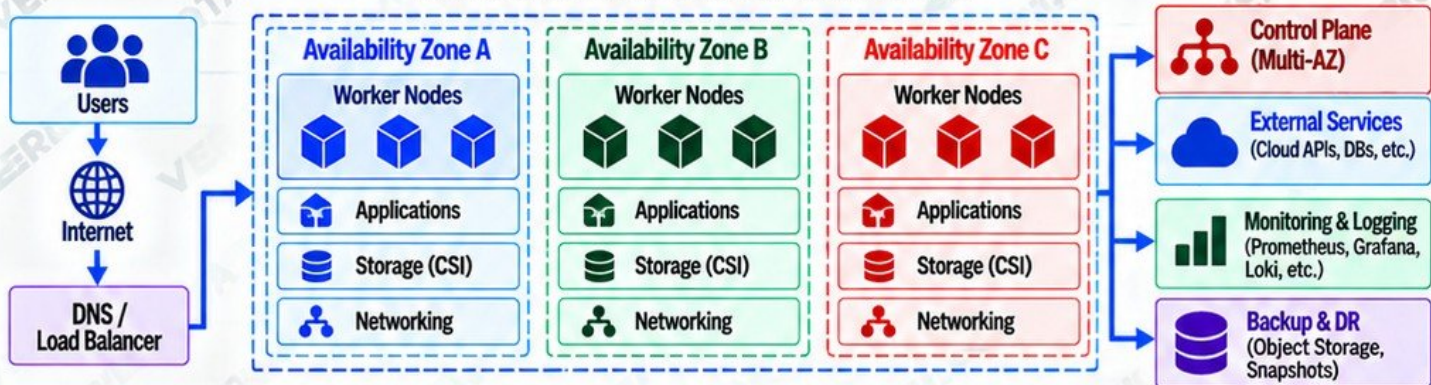


KEY MINDSET

- ✓ Stay calm and methodical
- ✓ Use data, not assumptions
- ✓ Communicate clearly
- ✓ Focus on user impact
- ✓ Turn every incident into a learning opportunity

Production Kubernetes Architecture Review

DESIGN, OPERATE AND SCALE A SECURE, RESILIENT AND COST-EFFICIENT KUBERNETES PLATFORM FOR PRODUCTION



1. MULTI-AZ TOPOLOGY



- Deploy control plane across multiple AZs (if self-managed).
- Distribute worker nodes across AZs.
- Use zonal or regional services.
- Avoid single points of failure.

Key Considerations:

- At least 3 AZs
- Zone failure testing
- Highly available control plane

2. WORKLOAD PLACEMENT



- Use node labels, taints and tolerations.
- Distribute workloads across AZs and node pools.
- Keep critical workloads separate.
- Avoid noisy neighbor issues.
- Use topology spread constraints.

Example:

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
```

3. CAPACITY HEADROOM



- Keep buffer capacity for spikes.
- Consider CPU, memory, storage and IP addresses.
- Plan for cluster growth.
- Monitor utilization and trends.
- Avoid running at 100%.

Key Targets:

- 20-30% headroom
- Separate system and user capacity
- Plan for scale-out time

4. NETWORK BOUNDARIES



- Use VPC and private subnets.
- Control ingress and egress.
- Use Ingress/Load Balancers.
- Restrict public exposure.
- Implement NetworkPolicies.
- Segment environments (dev, staging, prod).

Tools:

- VPC, subnets, security groups
- Ingress/NLB/ALB
- NetworkPolicy, service mesh (optional)

5. SECURITY BOUNDARIES



- Use RBAC and least privilege.
- Implement Pod Security Standards.
- Protect secrets (external manager).
- Use workload identity (cloud IAM).
- Restrict node access.
- Regular security scanning.

Key Controls:

- RBAC, Pod Security, NetworkPolicy
- Secrets management
- Image scanning and admission controls

6. FAILURE DOMAINS



- Understand what can fail (AZ, node, network, storage).
- Design for fault tolerance.
- Use replicas and anti-affinity.
- Test failure scenarios regularly.
- Document recovery procedures.
- Consider regional failures.

Examples:

- Node failure
- AZ outage
- Network partition
- Storage failure

7. UPGRADE STRATEGY



- Use a tested upgrade process.
- Follow Kubernetes version skew policy.
- Upgrade control plane first.
- Use rolling updates for nodes and add-ons.
- Test in a non-production environment.
- Have a rollback plan.

Best Practices:

- Read release notes
- Test add-on compatibility
- Upgrade in stages
- Monitor after upgrade

8. BACKUP AND RESTORE



- Back up etcd (if self-managed).
- Use Velero or cloud-native backups for resources.
- Back up persistent volumes (PV).
- Regularly test restore procedures.
- Document RTO and RPO.

Tools:

- Velero
- Cloud snapshots (EBS, PD, etc.)
- etcd backups
- Regular restore testing

9. DISASTER RECOVERY



- Plan for regional failures.
- Have a DR cluster (warm or cold).
- Use cross-region backups.
- Document failover and failback steps.
- Test DR plan regularly.
- Consider data replication and DNS failover.

Key Elements:

- DR region/cluster
- Data replication
- Tested runbooks
- Clear RTO and RPO

10. SLOs AND ALERTING



- Define SLOs for key services.
- Monitor availability, latency, error rate and saturation.
- Set up meaningful alerts.
- Use multi-window, multi-burn alerts.
- Reduce noise and alert fatigue.
- Review and tune regularly.

Example SLO:

- 99.9% availability
- < 300ms p95 latency
- < 1% error rate

11. COST VS RESILIENCE



- Right-size resources.
- Use autoscaling (HPA, VPA, Cluster Autoscaler).
- Use spot/preemptible nodes where possible.
- Balance cost savings with reliability.
- Avoid over-provisioning.
- Review costs regularly (FinOps).
- Consider multi-AZ cost trade-offs.

Tips:

- Use requests and limits
- Monitor unused resources
- Use spot nodes for non-critical workloads
- Measure cost per service

12. PRODUCTION-READINESS CHECKLIST



- ✓ Multi-AZ and HA architecture
- ✓ Workload placement and anti-affinity
- ✓ Capacity headroom and scaling plan
- ✓ Network and security boundaries
- ✓ Backup and restore tested
- ✓ Disaster recovery plan
- ✓ Upgrade strategy and rollback plan
- ✓ SLOs and alerting in place
- ✓ Cost optimization review
- ✓ Runbooks and documentation
- ✓ Regular testing (failover, restore, upgrades)
- ✓ Approved for production